

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 01 — ¿Qué es un agente de IA?

"Un asistente responde. Un agente percibe, planifica y actúa. La diferencia no es filosófica: es arquitectónica."

Objetivos de aprendizaje

- Establecer una definición operativa precisa de "agente de IA" que distinga el agente de un asistente conversacional y de una cadena simple de llamadas a herramientas.
- Comprender qué hace que un sistema sea agéntico y qué condiciones debe cumplir para merecer ese nombre.
- Ubicar el agente dentro de la arquitectura de Context Engineering construida a lo largo del módulo.
- Reconocer los límites conceptuales de este capítulo respecto a los capítulos adyacentes sobre herramientas (07) y sistemas multiagente (09).

El problema de la definición

La industria usa la palabra "agente" de maneras distintas y a veces contradictorias. LangChain llama agente a cualquier cadena que usa herramientas. AutoGPT popularizó la imagen de un sistema que se ejecuta indefinidamente sin supervisión. Los papers académicos definen agentes en términos de percepción, estado y función de utilidad. El SDK de Claude describe agentes como sistemas que orquestan múltiples herramientas y ciclos de razonamiento.

Antes de estudiar cómo se construye un agente, es necesario acordar qué entendemos por uno. Una definición imprecisa produce diseños imprecisos.

Definición operativa

Para este módulo, un agente de IA es un sistema que:

- 1. **Percibe un objetivo** definido por el usuario u otro sistema.
- 2. **Planifica una secuencia de acciones** para alcanzar ese objetivo.
- 3. **Ejecuta esas acciones** mediante herramientas externas, bases de conocimiento o llamadas a otros sistemas.
- 4. **Observa los resultados intermedios** de cada acción.
- 5. **Adapta su plan** en función de esos resultados hasta completar el objetivo o determinar que no puede completarlo.

Esta definición incluye cuatro elementos que lo distinguen de otros sistemas:

Característica	Asistente conversacional	Cadena de herramientas	Agente
Responde a una consulta	Sí	Sí	Sí
Ejecuta herramientas	No / limitado	Sí (orden fijo)	Sí (orden dinámico)
Adapta el plan según resultados	No	No	Sí
Puede iterar múltiples ciclos	No	No	Sí
Decide cuándo terminó	No	No	Sí

El punto central es la adaptación dinámica. Una cadena de herramientas ejecuta pasos en un orden predefinido. Un agente decide en cada paso qué hacer a continuación, basándose en lo que ya observó. Esa capacidad de decisión iterativa es lo que define la arquitectura agéntica.

Lo que un agente no es

Un agente no es un asistente que usa herramientas. Si el sistema llama a una herramienta de búsqueda y entrega el resultado directamente, eso es un asistente con acceso a una herramienta. La estructura es estímulo-respuesta con un paso adicional. No hay planificación, no hay iteración, no hay adaptación.

Un agente no es sinónimo de autonomía total. El nivel de autonomía de un agente es una decisión de diseño. Algunos agentes consultan al usuario en cada paso crítico. Otros operan de manera completamente autónoma dentro de límites definidos. Ambos son agentes. La autonomía es una variable del diseño, no un atributo definitorio.

Un agente no requiere un framework específico. LangChain, LangGraph, AutoGen, CrewAI y el SDK de Claude implementan patrones de agentes, pero un agente puede implementarse directamente llamando a la API de un modelo de lenguaje en un bucle de control personalizado. El framework facilita la implementación; no constituye el agente.

Por qué los agentes emergen en este punto del módulo

Los capítulos anteriores construyeron los bloques que un agente necesita:

- **Capítulo 04 (Memoria):** Un agente necesita recordar qué hizo en pasos anteriores y qué resultados obtuvo. Sin memoria, cada ciclo empieza desde cero.
- **Capítulo 05 (Instrucciones del sistema):** El agente opera bajo políticas definidas en el system prompt. Esas políticas determinan qué puede hacer y qué no, cómo debe comportarse ante resultados inesperados y cuándo debe escalar a un humano.
- **Capítulo 06 (RAG):** Un agente puede recuperar conocimiento externo como parte de su ciclo de acción. La decisión de cuándo recuperar y qué recuperar es parte de su planificación.
- **Capítulo 07 (Herramientas):** Las herramientas son los efectores del agente. Sin herramientas, el agente solo puede razonar y generar texto; no puede actuar sobre el mundo.

Un agente es la arquitectura que integra todos estos bloques en un ciclo de control coherente.

Los límites de este capítulo

Este capítulo cubre el agente individual. La arquitectura de un único agente: cómo está compuesto, cómo opera su ciclo interno, cómo gestiona estado y memoria, cómo decide qué herramientas usar y cuándo terminar.

El capítulo 09 cubre sistemas multiagente. Qué ocurre cuando la tarea supera la capacidad de un único agente o requiere paralelismo y especialización. Cómo múltiples agentes se coordinan, se delegan trabajo entre sí y gestionan la coherencia colectiva.

El capítulo 10 cubre planificación y razonamiento. Los mecanismos cognitivos que el modelo de lenguaje usa para razonar. La distinción es importante: este capítulo se ocupa de cómo está construido el agente; el capítulo 10 se ocupa de cómo piensa.

Qué contiene este capítulo

Las secciones siguen esta progresión:

1. **¿Qué es un agente de IA?** — Esta sección: definición, límites conceptuales.
2. **De asistentes a agentes autónomos** — El salto cualitativo entre responder y actuar.
3. **Componentes fundamentales** — Los bloques que componen cualquier agente.
4. **Arquitecturas de agentes modernas** — Los patrones de referencia estables: ReAct, Plan-and-Execute, Reflection.
5. **Ciclo de percepción, planificación y acción** — El ciclo ReAct en detalle con ejemplo completo.
6. **Gestión del estado y memoria** — Qué debe recordar el agente y cómo lo organiza.
7. **Uso coordinado de herramientas y RAG** — Cómo decide el agente qué herramienta usar y cuándo recuperar conocimiento.
8. **Orquestación y toma de decisiones** — Cuándo actuar autónomamente y cuándo escalar.
9. **Patrones y anti-patrones** — Lo que funciona y lo que rompe sistemas reales.
10. **Caso de estudio empresarial** — Una implementación completa de principio a fin.
11. **Laboratorio práctico** — Diseño y traza del ciclo de un agente simple.
12. **Checklist del AI Engineer** — Lo que debe verificarse antes de desplegar.
13. **Resumen** — Las ideas centrales del capítulo.
14. **Autoevaluación** — Preguntas para verificar la comprensión.
15. **Transición al Capítulo 9** — El paso del agente individual al sistema multiagente.

Ideas clave

- Un agente de IA percibe un objetivo, planifica acciones, las ejecuta y adapta el plan según los resultados. Esa capacidad de adaptación iterativa es su característica definitoria.
 - La diferencia entre un asistente con herramientas y un agente no es de complejidad: es de estructura de control. El agente decide dinámicamente qué hacer a continuación.
 - La autonomía de un agente es una variable de diseño, no un atributo fijo. Los niveles de supervisión humana son decisiones de arquitectura.
 - El agente es la síntesis de los bloques construidos en los capítulos anteriores: memoria, instrucciones del sistema, RAG y herramientas integradas en un ciclo de control.
-

Transición hacia la siguiente sección

Definir qué es un agente exige entender de dónde viene. La distinción entre asistentes y agentes no surgió de una definición académica: emergió de la práctica, del intento de resolver tareas que los asistentes conversacionales no podían completar. La siguiente sección traza esa evolución y articula con precisión el salto cualitativo entre responder y actuar.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 02 — De asistentes a agentes autónomos

"La diferencia entre un asistente y un agente no es cuánto sabe. Es si puede actuar, observar y replanearse."

Objetivos de aprendizaje

- Comprender el salto cualitativo entre un asistente conversacional y un agente autónomo.
- Identificar qué capacidades nuevas aparecen en la transición y qué problemas de diseño traen consigo.
- Analizar casos concretos donde un asistente es insuficiente y un agente es la respuesta correcta.
- Reconocer los distintos niveles de autonomía como un espectro de decisiones de diseño.

El asistente conversacional: fortalezas y límites

Un asistente conversacional bien diseñado es una herramienta poderosa. Puede responder preguntas complejas, redactar documentos, analizar datos presentados en el contexto, resumir conversaciones largas y adaptar su tono al usuario. Todo esto lo hace dentro de un ciclo simple y determinista: el usuario envía un mensaje, el asistente responde.

Ese ciclo tiene una limitación estructural: el asistente no puede hacer nada que no sea responder. No puede ir a buscar información adicional si la que tiene es insuficiente. No puede ejecutar una secuencia de pasos y ajustarla según los resultados intermedios. No puede

completar una tarea que requiere múltiples acciones sobre sistemas externos. Puede describir cómo se haría, pero no puede hacerlo.

Esta limitación no es un defecto de implementación. Es una consecuencia de la arquitectura estímulo-respuesta.

Cuándo el asistente resulta insuficiente

Considerar la siguiente solicitud: "Analiza las ventas del último trimestre, identifica los tres productos con mayor caída respecto al trimestre anterior, y redacta un informe ejecutivo con las posibles causas y recomendaciones."

Un asistente puede hacer esto si los datos ya están en el contexto. Pero en una empresa real, esos datos están en un sistema de análisis de ventas. El asistente necesitaría:

1. Consultar el sistema de ventas para obtener los datos del trimestre actual.
2. Consultar el sistema para obtener los datos del trimestre anterior.
3. Calcular las diferencias y ordenar los productos por caída.
4. Posiblemente consultar registros adicionales para identificar causas.
5. Redactar el informe con ese análisis completo.

Cada paso depende del resultado del anterior. El paso 3 no puede ejecutarse sin los resultados de los pasos 1 y 2. Si el sistema de ventas devuelve un error en el paso 1, la respuesta correcta no es continuar con datos incompletos: es detectar el error, intentar una alternativa o reportar el problema.

Esta estructura secuencial, dependiente y adaptativa es exactamente lo que define un agente.

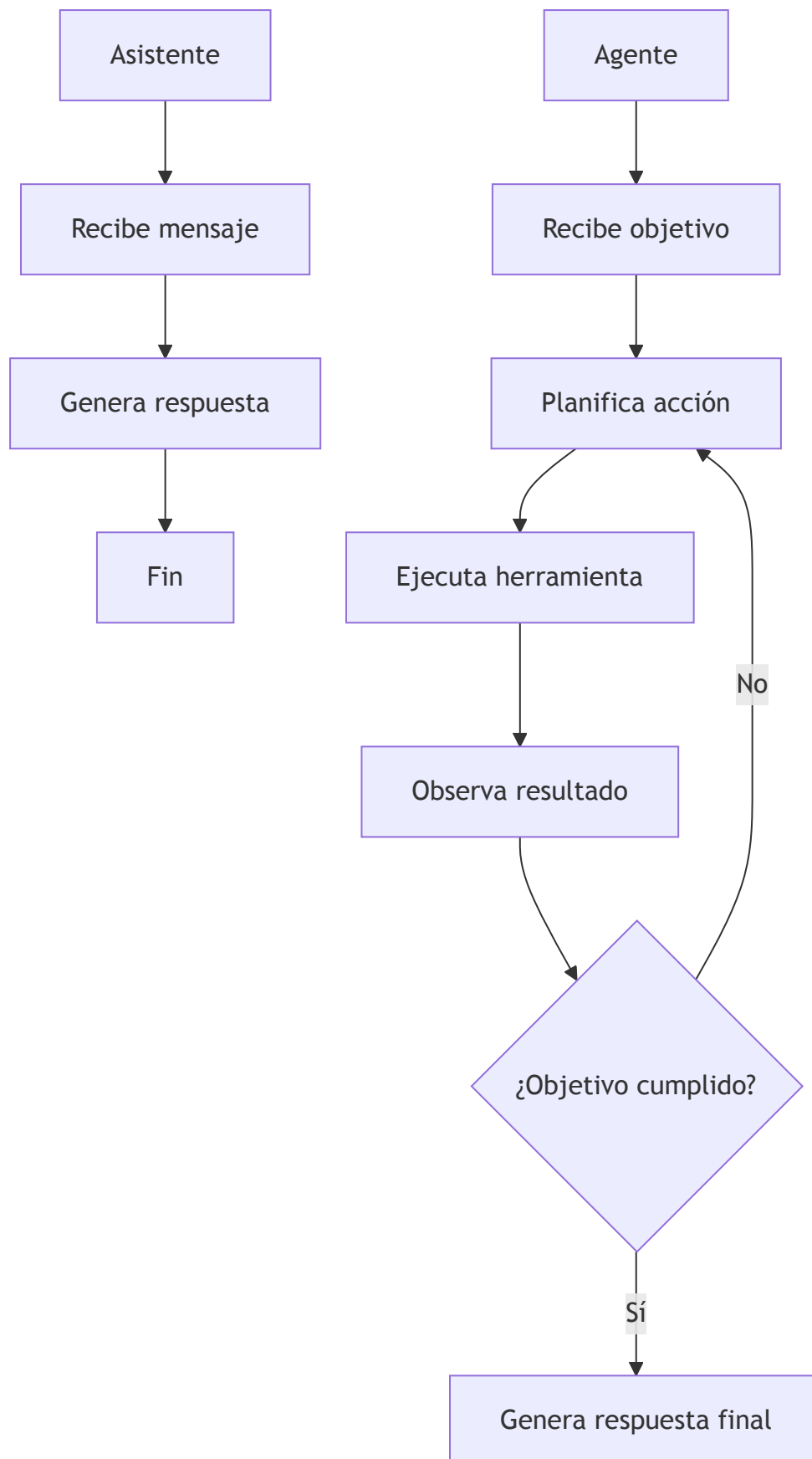
El salto cualitativo

La transición del asistente al agente introduce tres capacidades nuevas:

Acción sobre sistemas externos. El agente no solo genera texto: ejecuta herramientas que modifican el estado del mundo o recuperan información que no estaba en el contexto inicial. Una búsqueda en una base de datos, una llamada a una API, la escritura de un archivo: estas son acciones que el agente puede realizar.

Observación de resultados intermedios. Después de cada acción, el agente recibe una observación: el resultado de la herramienta, el error que produjo, los datos que devolvió. Esa observación se incorpora al razonamiento del siguiente paso.

Replanificación basada en observaciones. Si una acción no produce el resultado esperado, el agente puede cambiar de estrategia. No está obligado a seguir el plan original si el plan ya no funciona. Esta capacidad de adaptación es la que distingue al agente de una cadena de herramientas con orden fijo.



La diferencia visual es clara: el asistente tiene un camino lineal. El agente tiene un bucle. Ese bucle es el corazón de la arquitectura agéntica.

El espectro de autonomía

La autonomía de un agente no es un interruptor binario. Es un espectro con múltiples posiciones:

Nivel	Descripción	Ejemplo
0 — Asistente	Responde sin actuar	Chatbot de preguntas frecuentes
1 — Asistente con herramientas	Ejecuta una herramienta por consulta	Búsqueda web integrada en el chat
2 — Agente supervisado	Múltiples herramientas, confirma cada acción	Agente de soporte que requiere aprobación
3 — Agente semi-autónomo	Actúa libremente en pasos seguros, escala en pasos críticos	Agente de análisis que escala solo al modificar datos
4 — Agente autónomo	Opera sin intervención dentro de límites definidos	Pipeline de procesamiento de datos nocturno

Ningún nivel es mejor que otro por defecto. El nivel correcto depende del riesgo de las acciones, la reversibilidad de los errores, el costo de la supervisión y la confianza en el sistema.

En aplicaciones empresariales, los niveles 2 y 3 son los más comunes. La autonomía total (nivel 4) se reserva para procesos donde los errores son detectables y reversibles, o donde el costo de la supervisión humana supera con creces el riesgo de error.

Qué cambia en el diseño

La transición del asistente al agente no es solo agregar herramientas a un prompt. Es un cambio de paradigma de diseño que afecta múltiples dimensiones:

Diseño de la interacción. El usuario ya no espera una respuesta inmediata. Espera el resultado de un proceso que puede tomar múltiples pasos y un tiempo no despreciable. El

diseño de la experiencia debe reflejar ese proceso.

Gestión de errores. Un asistente que no puede responder genera una respuesta de error. Un agente que no puede completar una tarea debe detectar cuándo está bloqueado, reportar el estado en que quedó y ofrecer opciones de recuperación.

Trazabilidad. Cada acción del agente deja una huella. En producción, es necesario poder auditar qué hizo el agente, en qué orden, con qué herramientas y qué resultados obtuvo. Sin trazabilidad, depurar fallos o explicar el comportamiento del sistema es prácticamente imposible.

Límites y condiciones de terminación. Un agente sin condiciones de terminación bien definidas puede operar indefinidamente, agotando recursos o entrando en bucles. Establecer cuándo el agente debe declarar éxito, cuándo debe declarar fracaso y cuándo debe escalar es una decisión de diseño crítica.

Nota del Arquitecto

La transición de asistente a agente amplía enormemente las capacidades del sistema, pero también amplía enormemente el espacio de fallos posibles. Un asistente que falla, falla de manera visible: genera una respuesta incorrecta o incompleta. Un agente que falla puede hacerlo silenciosamente: ejecutar una secuencia de acciones que producen un estado incorrecto sin que ningún paso individual genere un error observable. Diseñar agentes robustos requiere pensar sistemáticamente en todos los estados de fallo posibles antes de desplegar el sistema.

Ideas clave

- El asistente conversacional tiene una limitación estructural: opera dentro del ciclo estímulo-respuesta y no puede ejecutar acciones sobre sistemas externos ni adaptar un plan según resultados intermedios.
- El agente introduce tres capacidades nuevas: acción sobre sistemas externos, observación de resultados intermedios y replanificación basada en observaciones.

- La autonomía es un espectro de diseño, no un atributo fijo. El nivel correcto depende del riesgo, la reversibilidad y el costo de la supervisión.
 - La transición al agente cambia el paradigma de diseño: la interacción, la gestión de errores, la trazabilidad y las condiciones de terminación deben rediseñarse.
-

Transición hacia la siguiente sección

Comprender el salto cualitativo entre asistente y agente es el primer paso. El segundo es entender de qué partes está hecho un agente. La siguiente sección descompone la arquitectura de un agente en sus componentes fundamentales y explica qué hace cada uno.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 03 — Componentes fundamentales de un agente

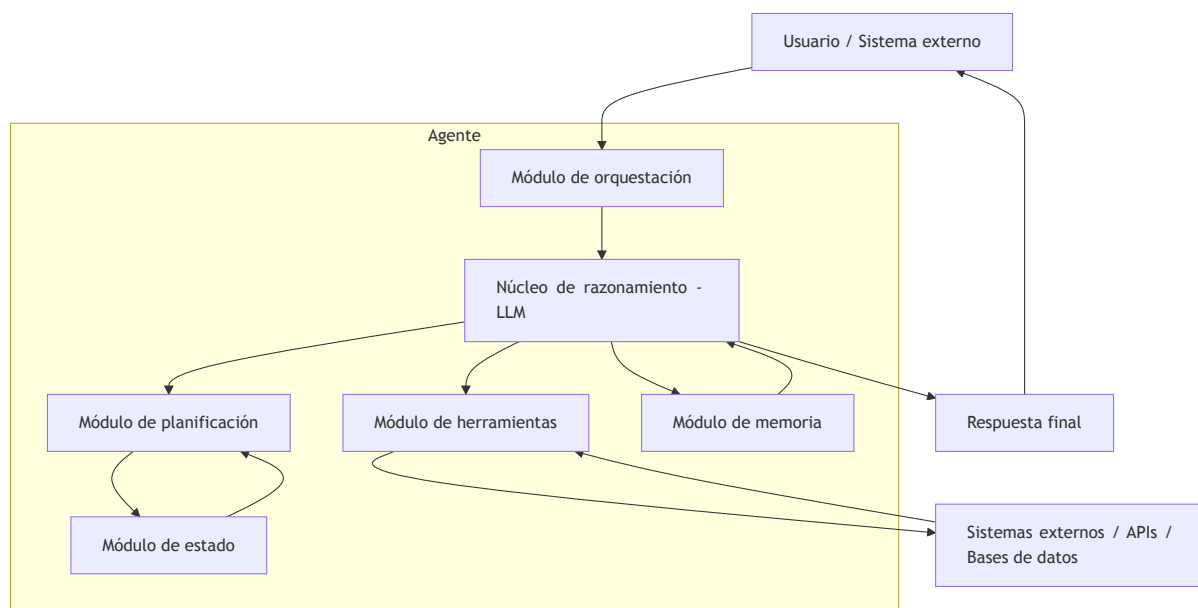
"Antes de diseñar un agente, es necesario saber de qué está hecho. Una arquitectura mal comprendida produce un sistema mal construido."

Objetivos de aprendizaje

- Identificar los componentes fundamentales que componen cualquier arquitectura de agente.
- Comprender el rol específico de cada componente y cómo interactúan entre sí.
- Relacionar cada componente con las capacidades desarrolladas en capítulos anteriores del módulo.
- Utilizar este mapa de componentes como referencia para las decisiones de diseño del resto del capítulo.

La arquitectura de referencia

Un agente de IA está compuesto por seis componentes que trabajan en conjunto. Este diagrama es la referencia visual del capítulo: cada sección posterior se enfoca en uno o más de estos componentes.



Los seis componentes son: el núcleo de razonamiento, el módulo de planificación, el módulo de estado, el módulo de herramientas, el módulo de memoria y la capa de orquestación.

Núcleo de razonamiento: el modelo de lenguaje

El modelo de lenguaje es el componente central del agente. No es una pieza más de la arquitectura: es el motor de todo el sistema. Es el que interpreta el objetivo, genera el plan, decide qué herramienta usar en cada paso, analiza los resultados y formula la respuesta final.

Esta centralidad del LLM es la característica definitoria de un agente basado en modelos de lenguaje. En sistemas de agentes más tradicionales (robótica, automatización de procesos), el módulo de razonamiento puede ser un sistema de reglas, un árbol de decisión o un planificador simbólico. En los agentes que estudiamos en este capítulo, el LLM asume ese rol.

Las implicaciones son importantes. El LLM no está programado explícitamente para cada decisión: razona en lenguaje natural y genera la siguiente acción como parte de ese razonamiento. Eso hace al agente flexible y capaz de generalizar, pero también introduce incertidumbre: el LLM puede cometer errores de razonamiento, malinterpretar herramientas o generar acciones inesperadas.

Módulo de planificación

El módulo de planificación estructura cómo el agente aborda el objetivo. En su forma más simple, la planificación ocurre dentro del propio razonamiento del LLM: el modelo genera un "pensamiento" sobre qué hacer a continuación antes de ejecutar la acción. En formas más elaboradas, el agente genera un plan completo al inicio y lo ejecuta paso a paso, ajustándolo según los resultados.

Los patrones de planificación se detallan en la sección 04. Lo que importa aquí es entender que la planificación no es opcional: todo agente que ejecuta más de una acción en secuencia está planificando, ya sea de forma explícita o implícita dentro del razonamiento del LLM.

Módulo de estado

El estado del agente es el registro de lo que ha ocurrido durante la ejecución actual. Incluye:

- El objetivo original recibido.
- Las acciones ejecutadas hasta el momento.
- Las herramientas invocadas y los parámetros usados.
- Los resultados observados de cada herramienta.
- El paso actual del plan.
- Los errores encontrados y cómo fueron manejados.

El estado es diferente de la memoria. El estado es local a una ejecución: cuando el agente termina su tarea, el estado de esa ejecución no necesariamente persiste. La memoria, en cambio, puede persistir entre ejecuciones y proporcionar continuidad entre sesiones. Esta distinción se desarrolla en detalle en la sección 06.

El estado también es la fuente principal del contexto que el LLM recibe en cada paso del ciclo. En cada iteración, el agente incluye en el prompt del LLM el historial de acciones y observaciones previas, de modo que el modelo pueda razonar teniendo en cuenta todo lo que ha ocurrido.

Módulo de herramientas

Las herramientas son los efectores del agente: los mecanismos mediante los cuales el agente actúa sobre sistemas externos. Sin herramientas, el agente solo puede razonar y generar texto;

no puede actuar sobre el mundo.

El capítulo 07 estudió en detalle cómo se definen, implementan y gestionan las herramientas. En el contexto del agente, el módulo de herramientas agrega una capa adicional: la decisión de qué herramienta usar en cada momento es parte del razonamiento del LLM, no un paso predefinido.

El LLM recibe la lista de herramientas disponibles con sus descripciones y firma, y decide cuál invocar en cada paso del ciclo. Esa decisión puede cambiar según los resultados observados. Si una herramienta falla, el agente puede intentar una herramienta alternativa. Si el resultado de una herramienta indica que se necesita información adicional, el agente puede invocar otra herramienta para obtenerla.

Módulo de memoria

El módulo de memoria proporciona continuidad al agente más allá del ciclo de ejecución actual. Puede almacenar y recuperar:

- Preferencias y características del usuario relevantes para interacciones futuras.
- Resultados de tareas anteriores que pueden ser reutilizados.
- Conocimiento acumulado sobre el dominio o el contexto del usuario.
- Resúmenes de conversaciones previas que sería costoso incluir completos.

La arquitectura de memoria para agentes sigue los mismos principios del capítulo 04, pero con una complicación adicional: el agente debe decidir qué información del estado actual merece persistir en la memoria y qué puede descartarse. Esa decisión de qué recordar es en sí misma parte del razonamiento del agente.

Capa de orquestación

La capa de orquestación coordina la ejecución del ciclo completo del agente. Es responsable de:

- Recibir el objetivo inicial y prepararlo para el primer ciclo.
- Invocar al LLM con el prompt correcto en cada iteración.

- Recibir la decisión del LLM (qué herramienta usar, con qué parámetros).
- Ejecutar la herramienta y capturar el resultado.
- Incorporar el resultado al estado y preparar el prompt del siguiente ciclo.
- Detectar las condiciones de terminación (objetivo cumplido, máximo de iteraciones alcanzado, error irrecuperable).
- Devolver la respuesta final al usuario o sistema que invocó al agente.

En implementaciones simples, la orquestación es un bucle de control explícito en el código de la aplicación. En frameworks como LangGraph o el SDK de Claude, la orquestación está abstraída en constructos de más alto nivel. Independientemente de la implementación, la lógica de orquestación siempre está presente.

Cómo interactúan los componentes en un ciclo

Para hacer tangible la interacción entre componentes, considerar un ciclo simple:

```
1. El usuario envía el objetivo: "Resume los tres contratos más recientes del cliente A"

2. La capa de ORQUESTACIÓN recibe el objetivo y prepara el primer prompt para el LLM.

3. El NÚCLEO DE RAZONAMIENTO (LLM) analiza el objetivo y genera:
    - Pensamiento: "Necesito buscar los contratos de Alfa. Usaré la herramienta buscar_c
    - Acción: buscar_contratos(cliente="Alfa", limite=10, orden="fecha_desc")

4. La ORQUESTACIÓN detecta la acción, invoca el MÓDULO DE HERRAMIENTAS.

5. El MÓDULO DE HERRAMIENTAS ejecuta la búsqueda y devuelve: 5 contratos con sus fechas

6. La ORQUESTACIÓN añade la observación al MÓDULO DE ESTADO y prepara el prompt para el

7. El LLM recibe el historial (objetivo + acción + observación) y razona:
    - "Ya tengo los contratos. Ahora necesito leer el contenido de los tres más reciente
    - Ejecuta tres llamadas secuenciales a leer_contrato(id=...) para cada uno.

8. Con el contenido de los tres contratos en el estado, el LLM genera la respuesta fina

9. La ORQUESTACIÓN detecta que el objetivo está cumplido y entrega la respuesta al usua
```

En este ejemplo simple ya están activos cinco de los seis componentes: razonamiento, planificación implícita, estado, herramientas y orquestación. La memoria no interviene porque la tarea no requiere información de sesiones anteriores.

Nota del Arquitecto

El diagrama de arquitectura de un agente es engañosamente simple. Seis cajas y algunas flechas no transmiten la complejidad de coordinar razonamiento, herramientas y estado en tiempo real. La dificultad está en los casos borde: qué ocurre cuando una herramienta devuelve un resultado ambiguo, cuando el LLM genera una acción que la herramienta no puede ejecutar, o cuando el estado crece

tanto que ya no cabe en la ventana de contexto. Diseñar cada componente por separado es necesario pero insuficiente: el comportamiento emergente de su interacción requiere pruebas de integración sistemáticas.

Ideas clave

- Un agente está compuesto por seis componentes: núcleo de razonamiento (LLM), módulo de planificación, módulo de estado, módulo de herramientas, módulo de memoria y capa de orquestación.
- El LLM es el motor central: decide qué herramienta usar, interpreta los resultados y genera la respuesta final.
- El estado es el registro local de la ejecución actual; la memoria proporciona continuidad entre ejecuciones. Son conceptos distintos con roles distintos.
- La capa de orquestación coordina el ciclo completo: desde el objetivo inicial hasta la respuesta final, gestionando las iteraciones y las condiciones de terminación.

Transición hacia la siguiente sección

Los componentes son los ladrillos. Las arquitecturas son los planos que indican cómo ensamblarlos. La siguiente sección estudia los patrones de arquitectura de agentes más utilizados en producción: sus supuestos, sus ventajas y las situaciones en que cada uno es la opción correcta.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 04 — Arquitecturas de agentes modernas

"Los patrones de arquitectura no son recetas. Son soluciones a problemas recurrentes, con supuestos explícitos sobre el contexto en que funcionan."

Objetivos de aprendizaje

- Conocer los patrones de arquitectura de agentes más utilizados en producción y sus fundamentos teóricos.
- Comprender los supuestos y limitaciones de cada patrón para poder elegir el correcto según el problema.
- Distinguir entre patrones estables, que pueden incluirse en un libro, y patrones en rápida evolución que requieren seguimiento continuo.
- Establecer criterios de selección de arquitectura para aplicaciones empresariales reales.

Por qué existen múltiples arquitecturas

No existe una arquitectura de agente universalmente óptima. Las diferentes arquitecturas surgieron como respuestas a problemas distintos: tareas que requieren planificación anticipada versus tareas donde el objetivo no puede determinarse de antemano; entornos donde los errores son costosos y reversibles versus entornos donde la velocidad es prioritaria; sistemas donde el LLM razona bien de forma autónoma versus sistemas donde el control explícito es necesario.

Conocer las arquitecturas disponibles no significa que deban implementarse todas. Significa poder elegir la correcta para cada problema.

Patrón ReAct: Reason + Act

ReAct es el patrón de agente más implementado en producción. Su nombre combina los dos pasos centrales del ciclo: razonamiento (Reason) y acción (Act).

En cada iteración del ciclo, el LLM genera tres elementos en secuencia:

- **Thought (pensamiento):** El modelo razona explícitamente sobre el estado actual, lo que ya sabe y lo que necesita hacer a continuación.
- **Action (acción):** El modelo especifica la herramienta a invocar y sus parámetros.
- **Observation (observación):** El resultado que devuelve la herramienta después de ser ejecutada. Este elemento lo genera el sistema, no el LLM, y se incorpora al prompt del siguiente ciclo.

```
Objetivo: "¿Cuál es el precio actual del dólar en Argentina y cómo se compara con el pr

--- Iteración 1 ---

Thought: Para responder necesito el precio actual del dólar. Usaré la herramienta de ti

Action: tipo_de_cambio(moneda="USD", mercado="oficial")

Observation: {"fecha": "2026-07-25", "valor": 1050.50, "moneda": "ARS"}

--- Iteración 2 ---

Thought: Tengo el precio actual. Ahora necesito el precio de hace una semana.

Action: tipo_de_cambio(moneda="USD", mercado="oficial", fecha="2026-07-18")

Observation: {"fecha": "2026-07-18", "valor": 1032.75, "moneda": "ARS"}

--- Iteración 3 ---

Thought: Tengo ambos valores. La variación es de 17.75 pesos (1.72%). Puedo responder.

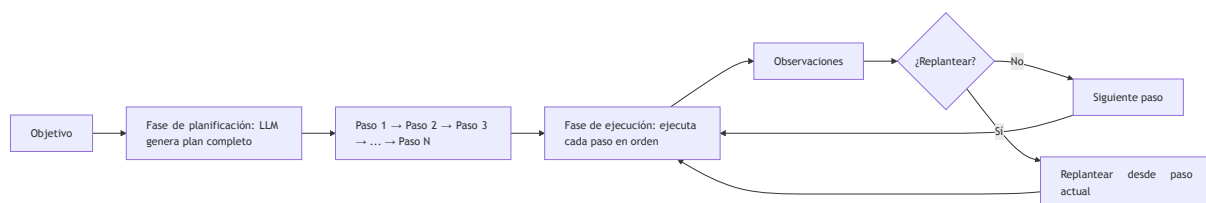
Action: [ninguna — el agente genera la respuesta final]

Respuesta: "El dólar oficial cotiza hoy a $1.050,50. Hace una semana valía $1.032,75, 1
```

La fortaleza de ReAct es su transparencia: el razonamiento explícito del agente es auditable. Los errores de razonamiento son visibles en el pensamiento antes de que produzcan una acción incorrecta. La debilidad es que el razonamiento explícito consume tokens adicionales y puede resultar redundante para tareas simples.

Patrón Plan-and-Execute

Plan-and-Execute separa la fase de planificación de la fase de ejecución. En la primera fase, el agente genera un plan completo para alcanzar el objetivo. En la segunda, ejecuta ese plan paso a paso.



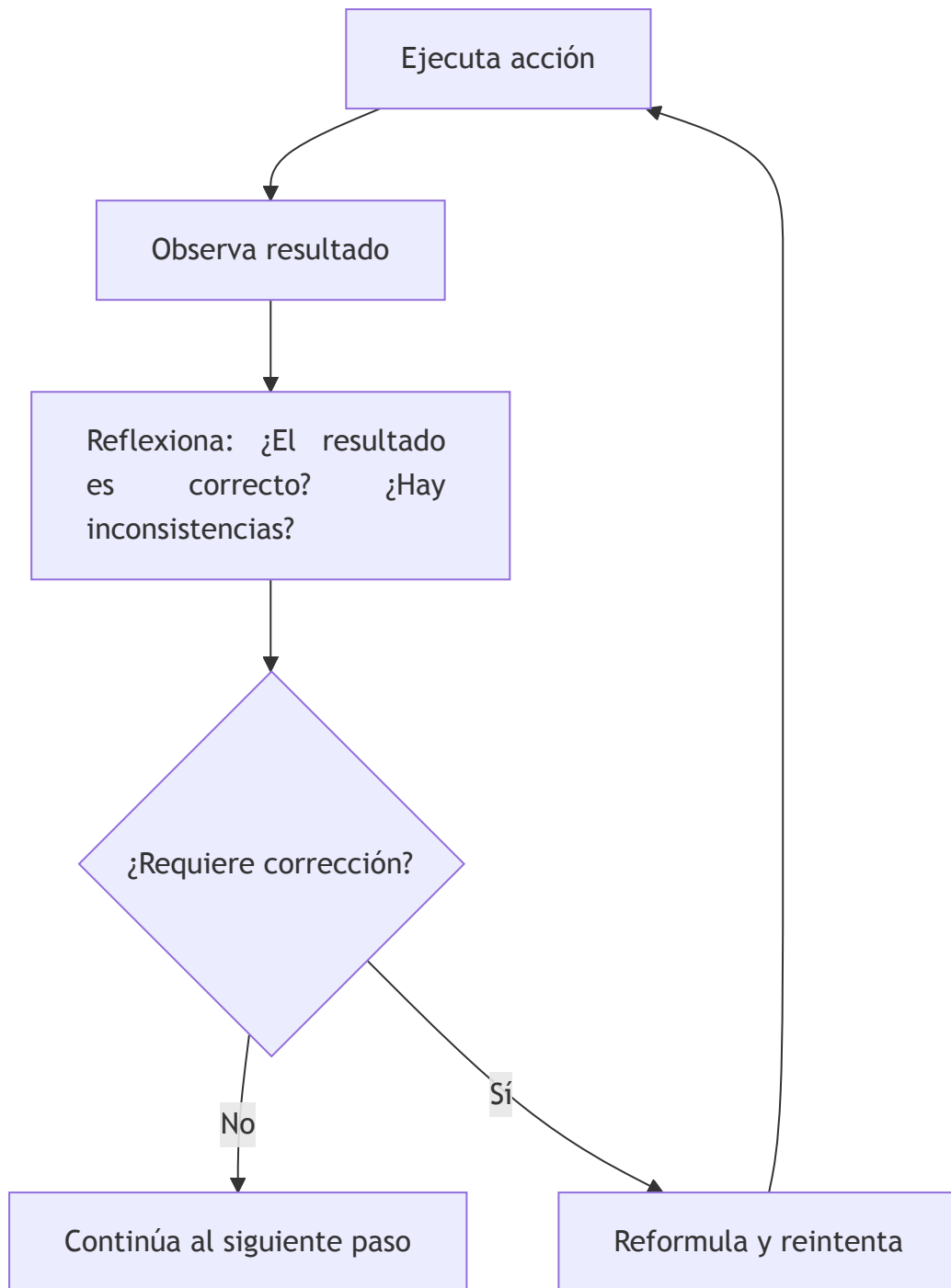
Este patrón es ventajoso cuando:

- El objetivo es complejo y puede descomponerse en pasos bien definidos antes de ejecutar.
- Se quiere reducir la latencia de razonamiento durante la ejecución (el plan ya fue generado).
- Es útil poder auditar el plan completo antes de autorizar la ejecución.

La limitación principal es que el plan inicial puede quedar obsoleto rápidamente si los primeros pasos devuelven resultados inesperados. La versión más robusta del patrón incluye un mecanismo de replanificación que revisa el plan restante cada vez que una observación difiere significativamente de lo esperado.

Patrón Reflection

Reflection agrega una etapa de autoevaluación después de cada acción o al final del ciclo. El agente no solo actúa y observa: también evalúa la calidad de sus propias acciones y resultados antes de continuar.



Reflection es especialmente útil para tareas donde la corrección del resultado es verificable internamente: generación de código (el código debe compilar y pasar tests), resolución de problemas matemáticos (el resultado puede verificarse con una herramienta de cálculo), o redacción de documentos (la coherencia interna puede evaluarse).

El costo es significativo: añade una llamada al LLM por cada ciclo de reflexión. Para tareas simples, ese costo no está justificado. Para tareas donde un error no detectado tiene

consecuencias graves, es una inversión razonable.

Patrón Self-Ask

Self-Ask es un patrón donde el agente descompone una pregunta compleja en subpreguntas más simples y las responde secuencialmente antes de sintetizar la respuesta final.

```
Pregunta: "¿Cuál es la empresa tecnológica con mayor crecimiento de ingresos en el últi  
Self-Ask:  
¿Necesito hacer una búsqueda? Sí.  
Sub-pregunta 1: ¿Cuáles son las principales empresas tecnológicas con sede en América I  
→ [Resultado de búsqueda]  
Sub-pregunta 2: Para cada empresa identificada, ¿cuál fue su crecimiento de ingresos en  
→ [Resultado para cada empresa]  
Sub-pregunta 3: ¿Cuál de esas empresas tiene el mayor crecimiento?  
→ [Comparación y selección]  
Respuesta final: [Síntesis basada en las respuestas a las sub-preguntas]
```

Self-Ask es efectivo para preguntas complejas donde el razonamiento correcto requiere responder primero preguntas intermedias. Su debilidad es que el agente puede formular subpreguntas incorrectas o innecesarias, especialmente si el objetivo es ambiguo.

Criterios de selección de arquitectura

Criterio	ReAct	Plan-and-Execute	Reflection	Self-Ask
Tarea con pasos predefinibles	Moderado	Alto	Bajo	Bajo
Tarea exploratoria (pasos dependen del resultado)	Alto	Bajo	Moderado	Moderado
Auditoría del proceso necesaria	Alto	Alto	Alto	Alto
Costo computacional	Moderado	Moderado	Alto	Moderado
Corrección verificable automáticamente	Bajo	Bajo	Alto	Bajo
Preguntas complejas con sub-preguntas	Moderado	Bajo	Bajo	Alto

Para la mayoría de las aplicaciones empresariales, ReAct es el punto de partida correcto. Es el patrón más documentado, el más implementado y el que mejor balancea capacidad y costo. La complejidad adicional de Plan-and-Execute o Reflection se justifica cuando el problema específico lo exige.

Una nota sobre la velocidad de cambio

El ecosistema de arquitecturas de agentes evoluciona rápidamente. Nuevos patrones aparecen regularmente en papers y frameworks. LATS (Language Agent Tree Search), ReWOO, Reflexion, y otros han sido propuestos y evaluados en los últimos años.

Los cuatro patrones presentados aquí tienen en común que son conceptualmente estables: sus principios no dependen de un framework específico, son implementables directamente y sus ventajas e inconvenientes son comprensibles y predecibles. Los patrones más recientes y especializados requieren seguimiento de la literatura técnica actualizada, ya que su madurez en producción es aún limitada.

Nota del Arquitecto

La elección de arquitectura no debe hacerse al inicio del proyecto, antes de conocer el problema en detalle. Empezar con ReAct y una implementación mínima proporciona

una línea base concreta contra la cual evaluar si la complejidad adicional de otro patrón está justificada. La optimización prematura de la arquitectura de agentes es uno de los errores más costosos en este tipo de proyectos.

Ideas clave

- ReAct (Reason + Act) es el patrón de agente más utilizado en producción. Combina pensamiento explícito, acción y observación en un ciclo transparente y auditable.
- Plan-and-Execute separa la planificación de la ejecución, con ventajas para tareas donde el plan puede establecerse anticipadamente.
- Reflection añade autoevaluación al ciclo, aumentando la robustez a costo de mayor consumo computacional.
- Self-Ask descompone preguntas complejas en sub-preguntas para mejorar el razonamiento encadenado.
- El criterio de selección principal es el tipo de tarea: exploratoria versus predefinible, verificable versus no verificable, simple versus compleja.

Transición hacia la siguiente sección

Los patrones de arquitectura describen la estructura del ciclo del agente. La siguiente sección entra en el detalle de ese ciclo: cómo funciona exactamente el ciclo de percepción, planificación y acción en ReAct, con un ejemplo completo de tres iteraciones que muestra cada elemento del proceso.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 05 — Ciclo de percepción, planificación y acción

"El ciclo no es una metáfora. Es el mecanismo concreto por el que un agente convierte un objetivo en resultado."

Objetivos de aprendizaje

- Comprender en detalle el ciclo de percepción, planificación y acción en el patrón ReAct.
- Trazar un ciclo completo de tres iteraciones con todos sus elementos: pensamiento, acción y observación.
- Entender cómo el contexto del agente crece con cada iteración y qué implica para el diseño.
- Identificar las condiciones que determinan cuándo el ciclo debe terminar.

El ciclo como unidad de análisis

El ciclo de percepción, planificación y acción es la unidad operativa del agente. Cada iteración del ciclo produce una acción que acerca al agente al objetivo o genera información que refina la comprensión del problema. La suma de esas iteraciones es la ejecución completa de la tarea.

Entender el ciclo en detalle no es un ejercicio académico. Es el conocimiento que permite al AI Engineer diseñar el prompt del agente, dimensionar el presupuesto de tokens, definir las condiciones de terminación y depurar comportamientos inesperados.

Los tres elementos del ciclo ReAct

Cada iteración del ciclo ReAct tiene tres elementos:

Thought (pensamiento). El LLM razona sobre el estado actual antes de decidir una acción. Este pensamiento es texto explícito generado por el modelo como parte de su respuesta. No es un paso interno invisible: es una parte del output que puede ser leída, auditada y usada para diagnóstico.

Action (acción). El LLM especifica qué herramienta invocar y con qué parámetros. La acción es la consecuencia directa del pensamiento. En la mayoría de las implementaciones, la acción se genera en un formato estructurado (JSON o similar) que la capa de orquestación puede parsear y ejecutar.

Observation (observación). El resultado que devuelve la herramienta después de ser ejecutada. La observación es generada por el sistema, no por el LLM, y se añade al contexto del agente antes del siguiente ciclo. El LLM no puede predecir la observación: solo puede razonar sobre ella una vez que la recibe.

Un ciclo completo: ejemplo en tres iteraciones

Tarea asignada al agente: "Identifica al cliente con mayor volumen de compras en el último mes y redacta un correo de fidelización personalizado."

El agente tiene acceso a dos herramientas:

- `consultar_ventas(periodo, orden, limite)` — devuelve clientes ordenados por volumen de compras.
- `obtener_perfil_cliente(cliente_id)` — devuelve el perfil completo del cliente.

Iteración 1

[Estado del contexto]

Objetivo: "Identifica al cliente con mayor volumen de compras en el último mes y redacta un correo de fidelización personalizado."

[Respuesta del LLM]

Thought: Necesito identificar al cliente con mayor volumen de compras en el último mes. Tengo acceso a la herramienta consultar_ventas. La usaré con el período del mes actual, ordenada por volumen descendente, tomando solo el primero.

Action: consultar_ventas(periodo="2026-07", orden="volumen_desc", limite=1)

[Sistema ejecuta la herramienta]

Observation: {"cliente_id": "CLI-0892", "nombre": "Distribuidora Meridional S.A.", "volumen_compras": 847500, "moneda": "ARS", "cantidad_pedidos": 23}

Iteración 2

```
[Estado del contexto]

Objetivo: [mismo]

Iteración 1 – Thought: [anterior]

Iteración 1 – Action: consultar_ventas(...)

Iteración 1 – Observation: {"cliente_id": "CLI-0892", ...}


[Respuesta del LLM]

Thought: Ya identifiqué al cliente: Distribuidora Meridional S.A. (CLI-0892), con
$847.500 en 23 pedidos este mes. Para redactar un correo personalizado, necesito
más información sobre el cliente: su contacto, el tipo de productos que compra y
si tiene algún segmento especial asignado. Usaré obtener_perfil_cliente.

Action: obtener_perfil_cliente(cliente_id="CLI-0892")


[Sistema ejecuta la herramienta]

Observation: {"cliente_id": "CLI-0892", "nombre": "Distribuidora Meridional S.A.",
"contacto": "Lic. Patricia Moreno", "email": "p.moreno@meridional.com.ar",
"segmento": "Premium", "productos_principales": ["ferretería industrial", "EPP"],
"cliente_desde": "2019-03-15", "ciudad": "Rosario"}
```

Iteración 3

[Estado del contexto]

Objetivo: [mismo]

Iteración 1 — [completa]

Iteración 2 — [completa]

[Respuesta del LLM]

Thought: Tengo toda la información que necesito. El cliente es Distribuidora Meridional S.A., contacto Lic. Patricia Moreno, segmento Premium, cliente desde 2019. Sus compras principales son en ferretería industrial y EPP. Puedo redactar un correo que reconozca su actividad de este mes, mencione su antigüedad y ofrezca algo relevante para su categoría de productos.

Action: [ninguna — el agente genera la respuesta final directamente]

Respuesta final:

Asunto: Gracias por otro mes excepcional, Distribuidora Meridional

Estimada Lic. Moreno:

A lo largo de este mes, Distribuidora Meridional S.A. ha sido una vez más nuestro cliente con mayor actividad: 23 pedidos y \$847.500 en compras de ferretería industrial y EPP. Llevan más de siete años eligiéndonos, y cada mes lo confirman.

Como cliente Premium, queremos reconocer ese compromiso con acceso anticipado a nuestra nueva línea de EPP certificada que estará disponible a partir de agosto. Si desea coordinar una presentación personalizada para su equipo en Rosario, quedo a disposición.

Muchas gracias por su confianza sostenida.

Cómo crece el contexto en cada iteración

Uno de los aspectos más importantes del ciclo ReAct es que el contexto del agente crece con cada iteración. El prompt que el LLM recibe en la iteración 3 contiene el objetivo original más todos los pensamientos, acciones y observaciones de las iteraciones anteriores.

```
Iteración 1: Objetivo → ~150 tokens  
Iteración 2: Objetivo + Iter1 completa → ~400 tokens  
Iteración 3: Objetivo + Iter1 + Iter2 completas → ~700 tokens
```

Este crecimiento es predecible y debe tenerse en cuenta en el diseño. Para tareas que requieren muchas iteraciones o que devuelven observaciones muy extensas, el contexto puede aproximarse al límite de la ventana del modelo. Las estrategias de gestión de contexto del capítulo 04 se aplican aquí directamente: resumir observaciones largas, truncar resultados de herramientas o descartar pensamientos anteriores cuando ya no son relevantes.

Condiciones de terminación

El ciclo debe terminar. Un agente sin condiciones de terminación robustas puede entrar en bucles infinitos, agotar el presupuesto de tokens o acumular errores sin resolución. Las condiciones de terminación son:

Éxito: El LLM determina que el objetivo está cumplido y genera la respuesta final sin invocar ninguna herramienta adicional.

Máximo de iteraciones: La capa de orquestación interrumpe el ciclo al alcanzar un límite predefinido. Típicamente entre 5 y 20 iteraciones según la complejidad esperada de las tareas.

Error irrecuperable: Una herramienta falla repetidamente, los resultados son inconsistentes o el agente detecta que no puede alcanzar el objetivo con los recursos disponibles.

Solicitud de intervención humana: El agente determina que la situación requiere una decisión humana y detiene la ejecución para escalar.

Estas condiciones deben estar codificadas en la capa de orquestación, no delegadas exclusivamente al razonamiento del LLM. Confiar en que el LLM siempre detectará cuándo

debe detenerse es una fuente frecuente de fallos en sistemas de agentes.

Nota del Arquitecto

El crecimiento del contexto en cada iteración no es solo un problema de tokens: es un problema de calidad del razonamiento. Los LLMs grandes tienen ventanas de contexto extensas, pero su capacidad de atención a fragmentos distantes del contexto disminuye a medida que el contexto crece. Una observación de la iteración 1 puede recibir menos atención en la iteración 15 que en la iteración 2. Para tareas con muchas iteraciones, la estrategia de incluir un resumen del estado actual al inicio de cada prompt (en lugar del historial completo) puede mejorar tanto la calidad del razonamiento como el uso de tokens.

Ideas clave

- El ciclo ReAct tiene tres elementos en cada iteración: Thought (razonamiento explícito del LLM), Action (herramienta a invocar con parámetros) y Observation (resultado generado por el sistema).
- El contexto del agente crece con cada iteración porque incluye el historial completo de pensamientos, acciones y observaciones. Este crecimiento debe gestionarse activamente.
- Las condiciones de terminación deben estar codificadas en la capa de orquestación. El límite de iteraciones, la detección de errores irreversibles y los puntos de escalada son decisiones de diseño, no comportamientos emergentes.
- El pensamiento explícito del agente es su principal herramienta de diagnóstico. Leer la secuencia de pensamientos de un agente que falló casi siempre revela en qué punto del razonamiento ocurrió el error.

Transición hacia la siguiente sección

El ciclo de acción del agente describe cómo opera en una sola ejecución. Pero los agentes que operan en entornos reales deben mantener coherencia entre múltiples ejecuciones, recordar

el contexto del usuario y gestionar el estado interno de tareas largas. La siguiente sección estudia la gestión del estado y la memoria del agente.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 06 — Gestión del estado y memoria del agente

"El estado dice qué está haciendo el agente ahora. La memoria dice quién es el usuario y qué ocurrió antes. Confundirlos produce sistemas que olvidan lo que importa o recuerdan lo que no sirve."

Objetivos de aprendizaje

- Distinguir con precisión el estado del agente de la memoria persistente del usuario.
- Comprender qué información debe estar en el estado, qué en la memoria y qué puede descartarse.
- Analizar los desafíos de gestión de estado en tareas largas o que abarcan múltiples sesiones.
- Diseñar una estrategia de memoria para agentes que opere de forma eficiente dentro de la ventana de contexto.

Dos conceptos frecuentemente confundidos

En la arquitectura de un agente conviven dos formas distintas de información temporal: el estado de la ejecución actual y la memoria persistente. Tratarlos como equivalentes produce diseños defectuosos. Separarlos con claridad es la base de una arquitectura de agente coherente.

El estado del agente

El estado es el registro de lo que está ocurriendo en la ejecución actual del agente. Es específico a una tarea, comienza cuando el agente recibe el objetivo y termina cuando la tarea concluye o falla.

El estado contiene:

- El objetivo original recibido.
- El plan generado (si el patrón lo incluye explícitamente).
- El paso actual del plan en ejecución.
- El historial de acciones ejecutadas y sus parámetros.
- Las observaciones recibidas de cada herramienta.
- Los errores encontrados y cómo fueron manejados.
- Las decisiones tomadas y el razonamiento que las justificó.

El estado vive en el prompt del agente en cada iteración. La sección 05 mostró cómo ese prompt crece iteración a iteración: cada pensamiento, acción y observación anterior se añade al contexto. El estado es, operativamente, el contenido acumulado en ese contexto durante la ejecución.

La característica clave del estado es que es efímero. Cuando la tarea termina, el estado de esa ejecución no necesariamente se conserva. Puede archivarse para auditoría, puede resumirse y almacenarse en memoria, o puede descartarse completamente. La decisión depende del diseño del sistema.

La memoria persistente

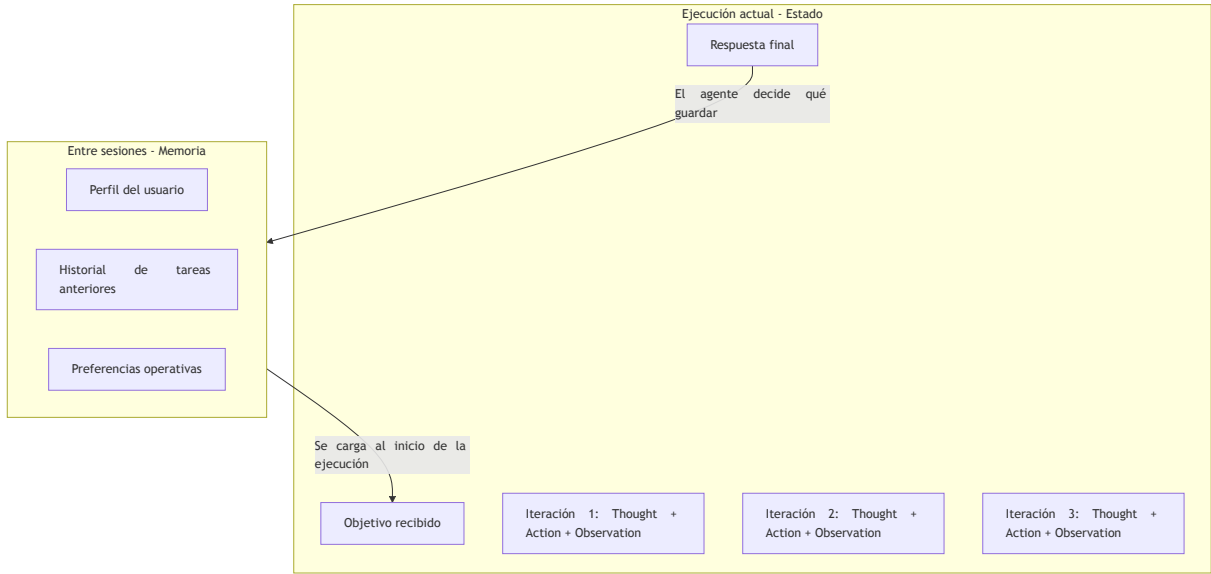
La memoria persistente es el mecanismo que permite al agente mantener continuidad entre ejecuciones separadas. Es lo que permite que el agente "recuerde" al usuario, sus preferencias, el contexto de interacciones anteriores y los resultados de tareas previas relevantes para la nueva.

El capítulo 04 desarrolló en detalle los tipos de memoria (episódica, semántica, procedimental) y las estrategias de gestión. En el contexto del agente, estos mecanismos se aplican con una particularidad: el agente no solo lee de la memoria, también decide qué escribir en ella como resultado de su ejecución.

Los tipos de información que suelen persistir en la memoria de un agente incluyen:

Tipo	Ejemplo	Vida útil típica
Perfil del usuario	Preferencias, rol, idioma	Larga (semanas a meses)
Resultados de tareas previas	"El análisis del Q2 se generó el 15/07"	Media (días a semanas)
Preferencias operativas	"El usuario prefiere respuestas concisas"	Larga
Contexto en curso	"El usuario está trabajando en el proyecto X"	Media
Errores conocidos	"La herramienta Y falla con este tipo de consulta"	Variable

La distinción en un diagrama



Al inicio de cada ejecución, el agente recupera de la memoria la información relevante para la tarea y la incluye en el contexto inicial. Al finalizar, decide qué información del estado merece persistir en la memoria para ejecuciones futuras.

El problema de qué recordar

Uno de los desafíos de diseño más delicados en agentes con memoria es decidir qué guardar. Guardar demasiado produce una memoria que crece sin control y degrada la relevancia de lo que se recupera. Guardar demasiado poco hace que el agente parezca olvidadizo.

Algunas heurísticas de diseño probadas:

Guardar resultados, no procesos. El detalle de cada iteración del ciclo no suele ser útil en futuras ejecuciones. El resultado final de la tarea, en cambio, puede ser muy relevante. Guardar "el análisis de ventas del Q2 fue completado el 15/07 con los datos del sistema CRM" es útil. Guardar cada pensamiento del ciclo que generó ese análisis generalmente no lo es.

Guardar hechos, no inferencias efímeras. Las inferencias que el agente realiza sobre el usuario en una sesión pueden ser incorrectas o estar sujetas a cambio. Los hechos explícitos ("el usuario indicó que trabaja en el departamento de finanzas") son más estables y útiles.

Aplicar TTL (time-to-live) a la información temporal. La información sobre el contexto en curso ("el usuario está preparando la presentación del Q3") tiene una vida útil corta. Diseñar la memoria con mecanismos de expiración evita acumular contexto obsoleto que puede confundir al agente.

Considerar la privacidad desde el diseño. La memoria de un agente puede contener información sensible. La arquitectura de memoria debe incluir desde el inicio mecanismos de control de acceso y políticas de retención que cumplan con las obligaciones legales y los requisitos del negocio.

Estado en tareas largas: el problema del contexto creciente

Las tareas largas plantean un problema específico de gestión de estado. Si la tarea requiere 30 iteraciones y cada iteración produce observaciones extensas, el contexto acumulado puede superar la ventana del modelo antes de que la tarea termine.

Las estrategias para manejar este problema son:

Compresión progresiva del historial. Después de un número determinado de iteraciones, el agente resume las iteraciones más antiguas y las reemplaza en el contexto por ese resumen. Las iteraciones recientes se mantienen completas. Esto preserva la continuidad del razonamiento sin exceder el límite de contexto.

Estado estructurado separado del historial. En lugar de incluir todo el historial de iteraciones en cada prompt, se mantiene un estado estructurado separado (un objeto JSON con los campos relevantes: objetivo, pasos completados, resultados clave, errores encontrados) y se actualiza en cada iteración. El prompt incluye el estado estructurado más solo las últimas N iteraciones completas.

Puntos de guardado intermedios. Para tareas muy largas, el agente puede definir hitos donde guarda el estado parcial en la memoria persistente y reinicia el contexto. Si algo falla después de un hito, el agente puede retomar desde el último punto guardado en lugar de empezar desde cero.

Nota del Arquitecto

La gestión del estado en agentes es una de las áreas donde los frameworks hacen más por el desarrollador y también donde más pueden ocultar problemas. LangGraph, por ejemplo, gestiona el estado como un grafo de nodos con transiciones explícitas, lo que hace el estado auditable y reutilizable. Pero si ese grafo no se diseña correctamente, puede acumular estado irrelevante o perder información crítica entre nodos. Independientemente del framework, es necesario poder responder estas preguntas: ¿qué está en el estado en cada iteración? ¿Qué se guarda en memoria al finalizar? ¿Qué se descarta? Si no se pueden responder, el diseño del estado es incompleto.

Ideas clave

- El estado del agente es efímero y específico a la ejecución actual. Contiene el objetivo, el historial de acciones y observaciones, y el razonamiento del ciclo en curso.
- La memoria persistente proporciona continuidad entre ejecuciones. Contiene información del usuario, resultados de tareas anteriores y preferencias operativas.
- El agente debe decidir activamente qué información del estado merece persistir en memoria. Guardar todo o no guardar nada son igualmente malos diseños.
- Las tareas largas requieren estrategias activas de compresión del historial o uso de estado estructurado para evitar que el contexto creciente degrade el razonamiento o exceda la

ventana del modelo.

Transición hacia la siguiente sección

El estado y la memoria determinan qué sabe el agente en cada momento. La siguiente sección estudia cómo el agente usa ese conocimiento para decidir qué hacer: la coordinación entre herramientas y RAG como los dos mecanismos principales de acción y recuperación de información en el ciclo del agente.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 07 — Uso coordinado de herramientas y RAG

"Un agente que tiene diez herramientas y no sabe cuándo usar cada una no es más capaz que uno que tiene dos y las usa correctamente."

Objetivos de aprendizaje

- Comprender cómo un agente decide qué herramienta usar en cada paso del ciclo.
- Distinguir cuándo el agente debe invocar una herramienta de acción versus cuándo debe recuperar conocimiento mediante RAG.
- Analizar los desafíos de coordinación cuando herramientas y RAG coexisten en el mismo ciclo.
- Diseñar una política de uso de herramientas que sea eficiente, predecible y segura.

Herramientas y RAG: dos mecanismos con roles distintos

El capítulo 07 estudió las herramientas como el mecanismo por el cual el agente actúa sobre sistemas externos. El capítulo 06 estudió RAG como el mecanismo por el cual el agente incorpora conocimiento externo al contexto. En el ciclo del agente, ambos coexisten, pero tienen roles distintos:

Mecanismo	Función principal	Produce
Herramienta de acción	Ejecuta algo en el mundo externo	Efecto y resultado (éxito/error/datos)
Herramienta de búsqueda	Recupera datos específicos de un sistema	Datos estructurados
RAG	Recupera conocimiento semántico de un corpus	Fragmentos de texto con contexto

La distinción no es absoluta. Una búsqueda en una base de datos es técnicamente una herramienta que recupera datos, pero su lógica es diferente a la de un sistema RAG que recupera fragmentos por similitud semántica. Ambos pueden coexistir en el mismo agente y el LLM debe decidir cuándo usar cada uno.

Cómo el LLM decide qué herramienta usar

La decisión de qué herramienta invocar emerge del razonamiento del LLM, no de una lógica de enrutamiento explícita. Esto es lo que distingue al agente de una cadena de herramientas: el agente decide dinámicamente en función del estado actual.

El mecanismo es simple en su formulación: el LLM recibe las descripciones de todas las herramientas disponibles y, como parte de su razonamiento, selecciona la que mejor responde a la necesidad del paso actual.

Herramientas disponibles:

- buscar_contratos(cliente_id, fecha_inicio, fecha_fin): Busca contratos de un cliente.
- leer_documento(doc_id): Recupera el contenido completo de un documento.
- buscar_conocimiento(consulta): Busca en la base de conocimiento por similitud semántica.
- enviar_email(destinatario, asunto, cuerpo): Envía un correo electrónico.
- crear_tarea(descripcion, responsable, fecha_limite): Crea una tarea en el sistema de gestión de tareas.

Objetivo: "El cliente CLI-0892 pregunta sobre las condiciones de renovación del contrato CONT-2024-087. Revisa el contrato, consulta nuestra política de renovaciones y prepara una respuesta para el cliente."

--- Iteración 1 ---

Thought: Primero necesito leer el contrato específico. Tengo su ID en el objetivo.

Action: leer_documento(doc_id="CONT-2024-087")

Observation: [contenido completo del contrato – cláusulas, fechas, condiciones]

--- Iteración 2 ---

Thought: Tengo el contrato. Ahora necesito conocer nuestra política de renovaciones.

Eso es conocimiento interno que está en nuestra base de documentación. Usaré

buscar_conocimiento porque no sé el ID exacto del documento de política.

Action: buscar_conocimiento(consulta="política de renovación de contratos condiciones p")

Observation: [tres fragmentos de la política de renovaciones con los términos relevantes]

--- Iteración 3 ---

Thought: Tengo el contrato y la política. El contrato vence el 31/10/2026, tiene cláusula de renovación automática con 90 días de preaviso para cancelar, y la política establece que clientes Premium tienen descuento del 5% en renovaciones. CLI-0892 es Premium. Puedo redactar la respuesta.

Action: [ninguna – genera respuesta final]

En este ejemplo, el agente usa `leer_documento` cuando conoce el ID exacto del recurso, y `buscar_conocimiento` (RAG) cuando necesita recuperar información por contenido sin saber exactamente qué documento contiene lo que busca. La decisión es parte del razonamiento del LLM.

La calidad de las descripciones de herramientas

La decisión del LLM sobre qué herramienta usar depende directamente de la calidad de las descripciones de las herramientas disponibles. Una descripción ambigua produce selecciones incorrectas. Una descripción precisa guía al modelo hacia la herramienta correcta.

Principios para describir herramientas en el contexto de un agente:

Describir el cuándo, no solo el qué. No basta con decir qué hace la herramienta. La descripción debe indicar en qué situaciones es la elección correcta y, cuando aplica, en qué situaciones no lo es.

```
# Descripción débil:
buscar_contratos - busca contratos en el sistema

# Descripción precisa:
buscar_contratos(cliente_id, fecha_inicio, fecha_fin) - Usa esta herramienta cuando
necesites encontrar contratos asociados a un cliente específico, filtrando por período.
Requiere el ID del cliente. No uses esta herramienta para recuperar el contenido de
un contrato: usa leer_documento(doc_id) con el ID del contrato que devuelve esta búsqueda
```

Especificar el formato de los parámetros. El LLM generará los parámetros de la herramienta como parte de su acción. Si el formato esperado es ambiguo, los parámetros generados pueden ser incorrectos.

Indicar qué devuelve la herramienta. El LLM razona sobre el resultado esperado para decidir si es la herramienta correcta. Si sabe que `buscar_contratos` devuelve IDs de contratos y no su contenido completo, entenderá que necesita un paso adicional con `leer_documento`.

Cuándo elegir RAG versus herramientas de búsqueda estructurada

Una de las decisiones de diseño más frecuentes en agentes empresariales es cuándo usar RAG y cuándo usar una herramienta de búsqueda estructurada.

Situación	RAG	Búsqueda estructurada
La consulta es semántica ("documentos sobre renovaciones")	Apropiado	No aplica
La consulta es por clave exacta ("contrato ID=CONT-2024-087")	No es la opción principal	Apropiado
El contenido está en documentos no estructurados	Apropiado	Limitado
El contenido está en una base de datos relacional	No aplica	Apropiado
Se necesita diversidad de perspectivas sobre un tema	Apropiado	Limitado
Se necesita un valor exacto o un conjunto acotado	No es la opción principal	Apropiado

La regla práctica es que RAG es la elección cuando el agente necesita encontrar conocimiento por su contenido y la búsqueda exacta no es posible. La búsqueda estructurada es la elección cuando el agente conoce los criterios exactos de búsqueda y opera sobre datos estructurados.

En agentes empresariales maduros, ambos mecanismos coexisten: la búsqueda estructurada cubre los datos de los sistemas operacionales, y RAG cubre el conocimiento interno de la organización (políticas, procedimientos, documentación técnica).

El riesgo de la sobreutilización de herramientas

Un agente con acceso a muchas herramientas puede volverse impredecible. El LLM puede invocar herramientas innecesarias (por ejemplo, buscar información que ya está en el contexto), puede encadenar llamadas redundantes o puede generar parámetros incorrectos para herramientas que no necesitaba usar.

Las estrategias para mitigar este riesgo:

Proveer solo las herramientas necesarias para cada tarea. Un agente no necesita tener acceso a todas las herramientas del sistema todo el tiempo. En diseños modulares, el

conjunto de herramientas disponibles puede variar según el tipo de tarea o el paso del plan en ejecución.

Incluir en el system prompt una guía de uso. Una sección del system prompt del agente que explique la política de uso de herramientas (cuándo preferir RAG, cuándo usar búsqueda directa, en qué situaciones no se deben invocar herramientas) reduce las decisiones incorrectas.

Validar las llamadas antes de ejecutarlas. La capa de orquestación puede incluir una etapa de validación que verifica que los parámetros generados por el LLM son coherentes con la firma de la herramienta antes de ejecutarla.

Nota del Arquitecto

El diseño del catálogo de herramientas de un agente es tan importante como el diseño del propio agente. Un catálogo bien diseñado hace que el agente tome decisiones correctas naturalmente. Un catálogo mal diseñado produce errores difíciles de depurar porque el agente "racionalmente" elige la herramienta equivocada dada la información que tiene. Si un agente toma decisiones de herramientas incorrectas repetidamente, el primer lugar donde buscar el problema no es el prompt del agente: es la descripción de las herramientas.

Ideas clave

- Las herramientas de acción y RAG tienen roles distintos en el ciclo del agente. Las herramientas actúan sobre sistemas externos o recuperan datos estructurados. RAG recupera conocimiento por similitud semántica de un corpus no estructurado.
- La decisión de qué herramienta usar emerge del razonamiento del LLM, que se basa en las descripciones de las herramientas disponibles. La calidad de esas descripciones determina directamente la calidad de las decisiones.
- En agentes empresariales maduros, búsqueda estructurada y RAG coexisten, cada uno cubriendo un tipo diferente de información.

- Un catálogo de herramientas sobredimensionado aumenta la incertidumbre del agente. Proveer solo las herramientas necesarias para cada contexto mejora la precisión y la predictibilidad.
-

Transición hacia la siguiente sección

Las herramientas y RAG son los recursos que el agente usa en su ciclo. La siguiente sección estudia cómo el agente orquesta internamente el uso de esos recursos: cómo decide la secuencia de acciones, cuándo actuar autónomamente y cuándo detener la ejecución para solicitar intervención humana.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 08 — Orquestación y toma de decisiones

"Un agente que nunca se detiene a pedir confirmación no es autónomo: es imprudente. La autonomía controlada es la que produce valor en producción."

Objetivos de aprendizaje

- Comprender cómo el agente orquesta la secuencia de acciones dentro de su ciclo individual.
- Identificar los puntos de decisión donde el agente debe elegir entre actuar autónomamente o escalar a un humano.
- Diseñar una política de control que equilibre autonomía y supervisión según el riesgo de cada acción.
- Implementar condiciones de parada robustas que eviten los fallos más comunes de agentes en producción.

La orquestación del agente individual

Esta sección tiene un límite conceptual preciso: cubre la orquestación interna del agente individual, es decir, cómo un único agente decide la secuencia de sus propias acciones. La coordinación entre múltiples agentes — cómo un agente orquestador delega trabajo a agentes especializados — es el tema del capítulo 09.

La orquestación interna del agente tiene tres dimensiones:

Qué hacer a continuación. En cada iteración, el agente decide la próxima acción basándose en el objetivo, el estado actual y las observaciones recibidas. Esta decisión es parte del razonamiento del LLM y emerge del ciclo ReAct estudiado en la sección 05.

Actuar autónomamente o escalar. En ciertos puntos del ciclo, el agente debe evaluar si la acción que está a punto de tomar requiere confirmación humana o si puede proceder de forma autónoma.

Cuándo declarar que terminó. El agente debe reconocer cuándo el objetivo está cumplido, cuándo no puede cumplirlo y cuándo la situación requiere intervención externa.

La decisión de cuándo escalar

En aplicaciones empresariales, los agentes rara vez operan con autonomía total. Existen categorías de acciones donde la supervisión humana no es opcional: no por limitación del sistema, sino por diseño deliberado.

Las categorías que típicamente requieren confirmación antes de proceder:

Acciones irreversibles. Eliminar datos, enviar comunicaciones a clientes, ejecutar transacciones financieras, modificar configuraciones de sistemas en producción. Una acción irreversible incorrecta puede tener consecuencias que no pueden deshacerse. La confirmación previa es la única salvaguarda disponible.

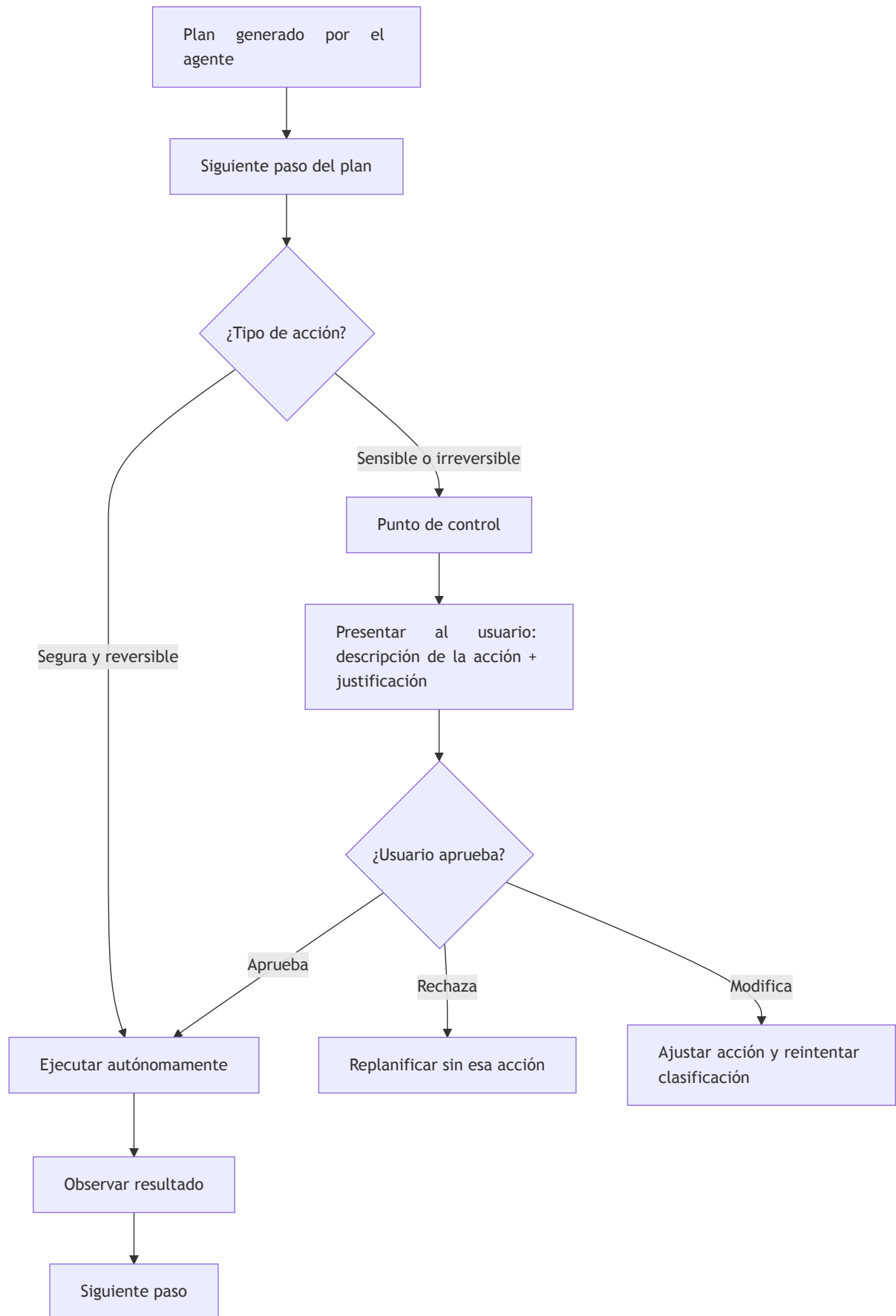
Acciones que afectan a terceros. Enviar un email en nombre de la organización, crear un registro en un sistema externo, publicar contenido. El agente actúa con la autoridad de la organización; esa autoridad debe ser validada antes de ejercerse.

Acciones en situaciones ambiguas. Si los datos disponibles no son suficientes para tomar una decisión con certeza razonable, el agente debe reportar la ambigüedad en lugar de adivinar. Una decisión incorrecta tomada con información insuficiente es más costosa que la demora de solicitar aclaración.

Acciones fuera del alcance definido. Si el objetivo del usuario o el contexto de la tarea cae fuera de los límites definidos en el system prompt, el agente debe escalar en lugar de improvisarse competencias que no le fueron asignadas.

Implementando los puntos de control

Los puntos de control son el mecanismo técnico para implementar la política de escalada. Un punto de control es un paso en el ciclo del agente donde la ejecución se pausa y espera confirmación antes de continuar.



La clasificación de acciones puede implementarse de varias maneras:

Clasificación estática. Se define en el diseño del sistema qué herramientas o tipos de acciones son siempre sensibles. Antes de invocar cualquiera de ellas, el agente inserta un punto de control. Es simple de implementar y predecible.

Clasificación dinámica. El agente evalúa el riesgo de cada acción en función de su contexto específico. "Enviar email al equipo interno" puede ser autónomo; "enviar email al cliente" puede requerir confirmación. Esta clasificación es más precisa pero más compleja de implementar.

En la mayoría de los sistemas empresariales, una combinación de ambas es la solución correcta: las herramientas siempre sensibles tienen puntos de control estáticos, y las herramientas de riesgo variable tienen clasificación dinámica.

El problema del agente que no para

Uno de los fallos más documentados en sistemas de agentes es el bucle infinito: el agente continúa iterando sin alcanzar el objetivo, sin detectar que está bloqueado y sin declarar un error. Este comportamiento puede ocurrir por varias razones:

- Una herramienta devuelve resultados que el agente interpreta como parciales y continúa buscando información que no existe.
- El agente genera una acción que falla, intenta alternativas que también fallan, y no tiene criterio para declarar el fallo como irrecuperable.
- El objetivo es ambiguo y el agente no puede determinar cuándo está cumplido.

Las salvaguardas contra el bucle infinito deben estar en la capa de orquestación, no en el razonamiento del LLM:

Límite absoluto de iteraciones. El agente nunca puede ejecutar más de N ciclos, independientemente de si ha alcanzado el objetivo. El valor de N debe calibrarse para cada tipo de tarea; un rango razonable para la mayoría de las aplicaciones empresariales es entre 10 y 25 iteraciones.

Detección de bucle. Si el agente ejecuta la misma acción con los mismos parámetros más de una vez, la orquestación detecta el patrón y interrumpe el ciclo.

Timeout absoluto. Independientemente de las iteraciones, la ejecución no puede superar un tiempo máximo. Esto protege contra casos donde las herramientas responden muy lentamente.

Presupuesto de tokens. Si el costo acumulado de la ejecución supera un umbral definido, la orquestación interrumpe el ciclo y reporta el estado parcial.

Diseñar la respuesta de fallo

Un agente que no puede completar su objetivo debe terminar gracefully. La respuesta de fallo debe incluir:

- Qué se intentó hacer.
- En qué punto del proceso se encontró el obstáculo.
- Por qué no se pudo continuar.
- Qué información parcial se obtuvo que pueda ser útil.
- Qué opciones tiene el usuario para continuar (escalar, reformular el objetivo, proveer información adicional).

Una respuesta de fallo bien diseñada convierte un fallo del agente en una interacción útil. El usuario entiende qué ocurrió y puede tomar una acción informada.

Nota del Arquitecto

La tentación en el diseño de agentes es maximizar la autonomía para minimizar la fricción con el usuario. Esa tentación produce sistemas que cometen errores costosos en nombre de la fluidez. La autonomía controlada no es una limitación del sistema: es una característica de diseño que distingue los agentes que funcionan en producción de los que funcionan en demos. En producción, la pregunta relevante no es "¿puede el agente hacer esto?", sino "¿debería el agente hacer esto sin confirmación?".

Ideas clave

- La orquestación interna del agente cubre tres decisiones: qué hacer a continuación, cuándo actuar autónomamente o escalar, y cuándo declarar que terminó.
- Los puntos de control son el mecanismo técnico para implementar la política de escalada. Deben aplicarse a acciones irreversibles, acciones que afectan a terceros y situaciones ambiguas.
- Las salvaguardas contra bucles infinitos (límite de iteraciones, timeout, detección de bucle) deben estar en la capa de orquestación, no en el razonamiento del LLM.
- Un agente que falla debe terminar gracefully, reportando el estado del proceso y las opciones disponibles para el usuario.

Transición hacia la siguiente sección

Conocer el funcionamiento interno del agente permite identificar qué diseños funcionan bien y cuáles producen problemas. La siguiente sección cataloga los patrones de diseño que los AI Engineers han validado en producción y los anti-patrones que explican los fallos más frecuentes en sistemas de agentes reales.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 09 — Patrones y anti-patrones de agentes

"Los anti-patrones no son errores de programadores inexpertos. Son errores de diseño que aparecen cuando la complejidad del sistema supera el modelo mental del equipo que lo construye."

Objetivos de aprendizaje

- Identificar los patrones de diseño que producen agentes robustos y mantenibles en producción.
- Reconocer los anti-patrones más frecuentes en sistemas de agentes y entender por qué surgen.
- Aplicar criterios concretos para evaluar si el diseño de un agente es sostenible a escala.
- Distinguir los fallos de diseño de los fallos de implementación.

Patrones que funcionan

Patrón: Herramientas atómicas

Las herramientas del agente deben hacer una sola cosa y hacerla bien. Una herramienta atómica tiene un input claro, un output predecible y no produce efectos secundarios fuera de su función declarada.

```
# Herramienta no atómica (problemática):
procesar_pedido(pedido_id) → busca el pedido, verifica stock, reserva inventario,
                           genera factura y envía email de confirmación

# Herramientas atómicas (preferible):
obtener_pedido(pedido_id) → devuelve los datos del pedido
verificar_stock(producto_id, cantidad) → devuelve disponibilidad
reservar_inventario(producto_id, cantidad, pedido_id) → realiza la reserva
generar_factura(pedido_id) → crea la factura
enviar_confirmacion(pedido_id, email) → envía el correo
```

Las herramientas atómicas permiten al agente construir flujos complejos combinando operaciones simples. Si algo falla, el fallo está localizado en una herramienta específica y es fácil de diagnosticar. Las herramientas monolíticas producen fallos difíciles de aislar y no pueden reutilizarse en otros contextos.

Patrón: Verificación antes de acción irreversible

Antes de ejecutar cualquier acción que no pueda deshacerse, el agente debe verificar explícitamente que tiene la información correcta y que la acción es la adecuada.

```
Thought: Voy a eliminar el registro CLI-0892 de la base de datos. Antes de proceder,
verifico: el objetivo del usuario fue "eliminar el cliente Alfa de la base". CLI-0892
corresponde a "Distribuidora Meridional S.A.", no a "Alfa". Hay una discrepancia.
No ejecutaré la eliminación. Solicitaré clarificación.
```

Este patrón requiere que el razonamiento del agente incluya explícitamente una etapa de verificación antes de acciones sensibles, y que el system prompt del agente establezca esta expectativa claramente.

Patrón: Terminación explícita

El agente debe generar explícitamente una declaración de terminación cuando considera que el objetivo está cumplido, en lugar de simplemente dejar de proponer acciones.

```
# Terminación implícita (ambigua):  
Thought: Creo que ya respondí la pregunta.  
[El agente genera texto sin invocar herramientas — la orquestación asume que terminó]  
  
# Terminación explícita (preferible):  
Thought: He completado los tres pasos del objetivo: identifiqué al cliente (CLI-0892),  
verifiqué su elegibilidad para la promoción (elegible, segmento Premium) y redacté el  
correo. El objetivo está cumplido.  
Action: finalizar_tarea(estado="exitoso", resumen="Cliente CLI-0892 identificado y  
correo de fidelización redactado.")
```

La terminación explícita produce sistemas más predecibles y más fáciles de depurar. La orquestación puede detectar la señal de terminación con certeza en lugar de inferirla.

Patrón: Fallback con información parcial

Cuando el agente no puede completar el objetivo, debe reportar cuánto avanzó antes del obstáculo, no simplemente fallar.

```
No se pudo completar el análisis de ventas porque el sistema CRM reportó un error  
de autenticación en la iteración 3. En ese punto ya se habían procesado los datos  
de enero y febrero (adjuntos). El análisis de marzo, abril y mayo no pudo completarse.  
Opciones para continuar: (1) reintentar en 30 minutos, (2) proveer las credenciales  
actualizadas del CRM, (3) continuar con los datos disponibles de enero y febrero.
```

Anti-patrones frecuentes

Anti-patrón: El agente todo-en-uno

Diseñar un agente que intenta manejar todos los casos de uso posibles de una aplicación en un único agente con docenas de herramientas. El resultado es un sistema con un prompt excesivamente largo, un catálogo de herramientas que el LLM no puede gestionar eficientemente y un comportamiento impredecible.

Por qué surge: Parece más simple tener un solo agente que manejar múltiples agentes especializados. El problema aparece a escala.

Consecuencias: El LLM toma decisiones de herramienta incorrectas, el contexto se agota antes de completar tareas complejas, y depurar fallos requiere analizar un sistema de gran complejidad.

Solución: Agentes especializados para dominios o tipos de tarea distintos, coordinados por un agente orquestador ligero. Este patrón se desarrolla en el capítulo 09.

Anti-patrón: Confianza ciega en el output de herramientas

El agente asume que el output de una herramienta es siempre correcto y completo, sin validación. Si la herramienta devuelve datos inconsistentes o parciales, el agente los incorpora al razonamiento sin cuestionar.

Ejemplo: Una herramienta de búsqueda devuelve `[]` (lista vacía). El agente razona "no hay resultados" y concluye que el cliente no tiene pedidos. En realidad, la herramienta falló silenciosamente porque la conexión a la base de datos se perdió.

Consecuencias: Respuestas incorrectas presentadas con confianza, sin indicación de que el resultado podría estar basado en datos incompletos.

Solución: Las herramientas deben distinguir explícitamente entre "no hay resultados" y "ocurrió un error". La orquestación debe validar que el output es coherente antes de pasarlo al agente.

Anti-patrón: El agente que improvisa herramientas

El agente intenta usar herramientas con parámetros o en combinaciones que no fueron diseñadas para ese uso, porque el LLM "razona" que puede funcionar.

Ejemplo: El agente necesita calcular un promedio de ventas. No tiene una herramienta de cálculo, pero tiene acceso a una herramienta de búsqueda SQL. Genera una consulta SQL ad hoc sin conocer el esquema exacto de la base de datos.

Consecuencias: La herramienta puede fallar, devolver resultados incorrectos, o en casos extremos, ejecutar operaciones no previstas.

Solución: El catálogo de herramientas debe cubrir las operaciones que el agente necesita. Si el agente improvisa sistemáticamente, el catálogo está incompleto. El system prompt debe incluir instrucciones explícitas sobre qué hacer cuando no existe la herramienta adecuada (declarar limitación, no improvisar).

Anti-patrón: Contexto sin gestión

El agente acumula todo el historial de iteraciones en el contexto sin ninguna estrategia de compresión o selección. En tareas largas, el contexto excede la ventana del modelo, se producen truncaciones silenciosas, o el costo por iteración se vuelve prohibitivo.

Consecuencias: Degradación del razonamiento (el LLM "olvida" información de iteraciones tempranas), fallos por exceso de tokens, o costos de inferencia que no escalan.

Solución: Implementar la estrategia de gestión de contexto diseñada en la sección 06: resúmenes progresivos del historial, estado estructurado separado del historial completo, puntos de guardado intermedios.

Anti-patrón: Herramientas con efectos secundarios ocultos

Una herramienta que parece de solo lectura en realidad modifica el estado de un sistema externo como efecto secundario. El agente llama a esa herramienta múltiples veces sin saber que cada llamada tiene consecuencias.

Ejemplo: `obtener_estado_pedido(pedido_id)` — parece una herramienta de lectura. En realidad, cada llamada registra un evento de "consulta" en el sistema de auditoría y puede disparar notificaciones automáticas si el estado es "pendiente de confirmación".

Consecuencias: El agente invoca la herramienta en múltiples iteraciones de diagnóstico sin saber que está generando efectos no deseados en el sistema externo.

Solución: Las herramientas deben documentar todos sus efectos, incluyendo efectos secundarios. Las herramientas de solo lectura deben serlo realmente. Si un efecto secundario es inevitable, debe documentarse explícitamente en la descripción de la herramienta.

Nota del Arquitecto

La mayoría de los anti-patrones no aparecen en la primera versión del agente. Aparecen cuando el sistema escala: más usuarios, más tipos de tarea, más herramientas, más casos borde. El momento de anticiparlos es en el diseño, no cuando ya están en producción. Una revisión estructurada del diseño usando esta lista de anti-patrones como checklist, antes del primer despliegue, evita la mayoría de los problemas costosos.

Ideas clave

- Los patrones de agentes robustos comparten tres características: herramientas atómicas, verificación antes de acciones irreversibles, y terminación explícita.
- Los anti-patrones más frecuentes son: el agente todo-en-uno (difícil de escalar), la confianza ciega en herramientas (produce errores silenciosos), la improvisación de herramientas (usa herramientas fuera de su diseño), el contexto sin gestión (degrada el razonamiento), y los efectos secundarios ocultos (produce consecuencias no previstas).
- Los anti-patrones surgen generalmente a escala, no en el prototipo inicial. El diseño debe anticiparlos.
- Un agente que falla gracefully, con información parcial y opciones de recuperación, es más valioso en producción que uno que falla silenciosamente.

Transición hacia la siguiente sección

Los patrones y anti-patrones son conocimiento abstracto hasta que se aplican a un caso real. La siguiente sección presenta un caso de estudio empresarial completo que integra todos los conceptos de este capítulo en una implementación de principio a fin.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 10 — Caso de estudio: Agente de soporte técnico empresarial

"Un caso de estudio bien construido hace visible lo que los conceptos dejan implícito. Los detalles de implementación son donde vive la ingeniería real."

Objetivos de aprendizaje

- Integrar los conceptos del capítulo en un diseño de agente empresarial completo y coherente.
- Analizar las decisiones de diseño concretas que produce cada concepto cuando se aplica a un caso real.
- Identificar los compromisos entre capacidad, costo, latencia y seguridad en un sistema de producción.
- Utilizar este caso como plantilla mental para abordar diseños de agentes en contextos similares.

Contexto del caso

Una empresa de software B2B ofrece soporte técnico a sus clientes a través de un sistema de tickets. El equipo de soporte recibe entre 200 y 400 tickets diarios. Aproximadamente el 60% de los tickets pueden resolverse con documentación existente; el 30% requiere consultar el historial técnico del cliente; el 10% requiere escalada a ingenieros.

El objetivo es construir un agente que maneje la primera atención de cada ticket: identifique la categoría del problema, recupere la información relevante, proponga una solución cuando sea posible, y escale cuando sea necesario. El agente opera en nombre de los agentes de soporte, no directamente con los clientes.

Definición del objetivo del agente

El agente recibe un ticket con los siguientes campos:

- `ticket_id` : identificador único del ticket.
- `cliente_id` : identificador del cliente que reportó el problema.
- `descripcion` : texto libre con la descripción del problema.
- `prioridad` : baja, media, alta, crítica.
- `producto` : módulo o producto afectado.

El agente debe producir:

- Categorización del problema (tipo: configuración, error de datos, bug conocido, integración, otro).
 - Historial relevante del cliente (tickets anteriores similares, soluciones aplicadas).
 - Solución propuesta o, si no existe solución disponible, diagnóstico del problema.
 - Recomendación de acción: resolver autónomamente, escalar al nivel 2, o escalar al equipo de ingeniería.
-

Herramientas disponibles

```
obtener_ticket(ticket_id)
```

→ Devuelve todos los campos del ticket.

```
obtener_historial_cliente(cliente_id, limite, tipo_filtro)
```

→ Devuelve los últimos N tickets del cliente, con filtro opcional por tipo o producto.

```
buscar_en_documentacion(consulta)
```

→ RAG sobre la base de conocimiento técnico: manuales, guías de resolución, release notes.

```
buscar_bugs_conocidos(producto, descripcion)
```

→ Búsqueda en el registro de bugs conocidos con coincidencia por producto y descripción.

```
obtener_configuracion_cliente(cliente_id, producto)
```

→ Devuelve la configuración actual del cliente para un producto específico.

```
crear_resolucion(ticket_id, categoria, solucion, accion_recomendada)
```

→ Registra la resolución propuesta en el sistema de tickets. ACCIÓN IRREVERSIBLE.

```
escalar_ticket(ticket_id, nivel, justificacion)
```

→ Escala el ticket al nivel especificado (nivel_2 o ingenieria). ACCIÓN IRREVERSIBLE.

Decisiones de diseño

Arquitectura: ReAct. El problema es exploratorio (los pasos dependen de la naturaleza del ticket) y el razonamiento transparente es valioso para los agentes de soporte que supervisan el sistema.

Punto de control: Las dos herramientas marcadas como irreversibles (`crear_resolucion` y `escalar_ticket`) siempre requieren un punto de control. El agente presenta la acción propuesta y espera confirmación del agente de soporte antes de ejecutarla.

Límite de iteraciones: 8 iteraciones máximo. Los tickets típicos se resuelven en 3-5 iteraciones. Si el agente necesita más de 8, el ticket es candidato a escalada.

RAG vs. búsqueda estructurada: `buscar_en_documentacion` usa RAG para recuperar guías y procedimientos por contenido semántico. `buscar_bugs_conocidos` y `obtener_historial_cliente` son búsquedas estructuradas sobre datos bien definidos.

Traza del ciclo: ticket de ejemplo

Ticket recibido:

```
ticket_id: TKT-20260725-1847
cliente_id: CLI-0445 (Comercializadora del Norte S.A.)
prioridad: alta
producto: módulo de facturación
descripcion: "Desde ayer no podemos generar facturas electrónicas. El sistema muestra error CFDI-002 al intentar timbrar. Necesitamos resolverlo urgente."
```

Iteración 1

```
Thought: Ticket de prioridad alta sobre un error específico (CFDI-002) en el módulo de facturación. Primero buscaré si este error es un bug conocido.
Action: buscar_bugs_conocidos(producto="facturacion", descripcion="CFDI-002")
Observation: {"bugs": [{"id": "BUG-1204", "codigo": "CFDI-002",
"descripcion": "Error de autenticación con el PAC cuando el certificado SAT vence o fue renovado sin actualizar en el sistema", "solucion": "actualizar el certificado en Configuración → Facturación → Certificados SAT",
"afecta_versiones": ["4.2", "4.3", "4.4"], "estado": "conocido_con_solucion"]}]}
```

Iteración 2

Thought: El error CFDI-002 tiene una solución conocida: el certificado SAT venció o fue renovado sin actualizar en el sistema. Necesito verificar la configuración actual del cliente para confirmar que este es el caso.

Action: obtener_configuracion_cliente(cliente_id="CLI-0445", producto="facturacion")

Observation: {"certificado_sat": {"vigencia": "2026-07-20", "estado": "vencido"}, "version_modulo": "4.3", "pac": "Finkok"}

Iteración 3

Thought: Confirmado. El certificado SAT de Comercializadora del Norte venció el 20/07/2026 (hace 5 días), y están en la versión 4.3 que está afectada por BUG-1204. La solución conocida aplica directamente. Debo consultar la documentación para dar instrucciones precisas de cómo actualizar el certificado.

Action: buscar_en_documentacion(consulta="cómo actualizar certificado SAT módulo facturación")

Observation: [fragmento: "Para actualizar el certificado SAT: 1) Ir a Configuración → Facturación → Certificados SAT. 2) Hacer clic en 'Actualizar certificado'. 3) Subir los archivos .cer y .key generados en el portal del SAT. 4) Ingresar la contraseña de la clave privada. 5) Verificar el nuevo período de vigencia y guardar. 6) Emitir una factura de prueba para confirmar la resolución."]

Iteración 4 — Punto de control

Thought: Tengo toda la información necesaria. El diagnóstico es claro (certificado vencido), la solución es conocida y la documentación confirma los pasos exactos. Voy a proponer la resolución. Esta acción requiere confirmación porque es irreversible.

Action: [PUNTO DE CONTROL – presentar al agente de soporte antes de ejecutar]

Propuesta de resolución para TKT-20260725-1847:

- Categoría: configuración
- Diagnóstico: Certificado SAT vencido el 20/07/2026. Error CFDI-002 es consecuencia de
- Solución: Instruir al cliente para que actualice el certificado en Configuración → Facturación → Certificados SAT (pasos detallados incluidos).
- Acción recomendada: resolver – nivel de soporte actual tiene toda la información necesaria
- Referencia: BUG-1204, documentación sección 3.2.

¿Confirma la resolución?

[Agente de soporte confirma]

```
Action: crear_resolucion(ticket_id="TKT-20260725-1847", categoria="configuracion",
solucion="Certificado SAT vencido el 20/07. Instrucciones de actualización enviadas al
Referencia: BUG-1204.",
accion_recomendada="resolver")

Observation: {"estado": "resolucion_registrada", "timestamp": "2026-07-25T14:32:07"}
```

Análisis de las decisiones de diseño en acción

Este ciclo ilustra varios principios del capítulo:

Herramientas atómicas: Cada herramienta hace una sola cosa. La verificación del bug, la verificación de la configuración del cliente y la búsqueda en documentación son operaciones separadas. Si alguna falla, el fallo está localizado.

RAG vs. búsqueda estructurada: `buscar_bugs_conocidos` y `obtener_configuracion_cliente` son búsquedas estructuradas con parámetros exactos. `buscar_en_documentacion` usa RAG porque el contenido es texto no estructurado y la consulta es semántica.

Punto de control antes de acción irreversible: El agente presenta la resolución propuesta antes de registrarla. El agente de soporte revisa y confirma. Esto evita registrar resoluciones incorrectas que podrían confundir al cliente.

Terminación explícita: La ejecución termina cuando la resolución es registrada, no cuando el agente deja de proponer acciones.

Cuatro iteraciones: El ticket se resolvió en 4 iteraciones (incluyendo el punto de control). El límite de 8 iteraciones no se alcanzó. Esto es indicativo de un diseño bien calibrado para el tipo de tarea.

Nota del Arquitecto

Este caso de estudio cubre el happy path: el ticket tiene un bug conocido con solución documentada y la configuración del cliente confirma el diagnóstico. En producción, el 40% de los tickets no siguen este camino. El diseño debe anticipar los casos donde el bug no está registrado, la documentación no tiene la respuesta, o la configuración del cliente es inconsistente. Para esos casos, el agente debe tener instrucciones explícitas en el system prompt sobre cuándo escalar y cómo reportar el estado parcial. El agente de soporte técnico del happy path es un buen primer paso; el agente que maneja bien los casos difíciles es el que produce valor en producción.

Ideas clave

- El diseño de un agente empresarial requiere definir explícitamente: el objetivo, las herramientas disponibles, la política de puntos de control, el límite de iteraciones y la estrategia de fallback.
- Los puntos de control para acciones irreversibles no son una limitación de autonomía: son el mecanismo que hace confiable al agente en contextos de negocio.

- La elección entre RAG y búsqueda estructurada debe reflejar la naturaleza de los datos: semántica para documentación no estructurada, búsqueda exacta para datos operacionales bien definidos.
 - Un ciclo de 4 iteraciones para un ticket de prioridad alta es un indicador de que el diseño está calibrado correctamente. Un ciclo de 8 iteraciones para el mismo tipo de ticket indicaría que algo en el diseño requiere revisión.
-

Transición hacia la siguiente sección

El caso de estudio describió el proceso de diseño y ejecución de un agente. El laboratorio de la siguiente sección invita al lector a trazar su propio ciclo: diseñar el flujo de un agente simple, identificar sus herramientas, definir sus condiciones de terminación y anticipar sus fallos.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 11 — Laboratorio práctico

"El diseño de un agente no se aprende leyendo sobre él. Se aprende trazando el ciclo paso a paso, con un objetivo real y herramientas concretas."

Objetivos del laboratorio

- Trazar el ciclo completo de un agente simple desde el objetivo hasta la respuesta final.
- Diseñar el catálogo de herramientas para una tarea específica, con descripciones precisas.
- Identificar los puntos de control y las condiciones de terminación correctas para el caso diseñado.
- Anticipar los fallos posibles y definir cómo el agente debe responder ante ellos.

Estructura del laboratorio

Este laboratorio no requiere código. El objetivo es diseñar y trazar, con el nivel de detalle suficiente para que otro ingeniero pueda implementar lo que se describe.

El laboratorio tiene cuatro ejercicios progresivos. Cada uno puede realizarse de forma independiente o en secuencia como un diseño completo.

Ejercicio 1 — Definir el agente

Contexto: Una empresa de logística quiere automatizar la primera etapa de gestión de reclamos de clientes por envíos retrasados. El agente debe: verificar si el envío está efectivamente retrasado, identificar la causa del retraso, y proponer al operador la acción apropiada (reembolso parcial, reembolso total, o escalada).

Tarea: Completar la ficha de definición del agente.

FICHA DE DEFINICIÓN DEL AGENTE

=====

Nombre del agente: _____

Objetivo principal:

[Describir en una oración qué debe lograr el agente al finalizar su ejecución]

Usuario/Sistema que activa el agente:

[¿Quién envía el objetivo inicial? ¿Es un humano o un sistema automatizado?]

Condiciones de éxito:

[¿Qué debe ser verdadero al finalizar para que el agente declare éxito?]

Condiciones de fallo:

[¿Cuándo debe el agente declarar que no puede completar el objetivo?]

Límite de iteraciones: _____ (justificar el valor elegido)

Nivel de autonomía: Supervisado / Semi-autónomo / Autónomo

[Indicar el nivel elegido y justificar por qué es apropiado para este caso]

Ejercicio 2 — Diseñar el catálogo de herramientas

Tarea: Para el agente del ejercicio 1, diseñar entre 4 y 6 herramientas. Para cada herramienta, completar la ficha.

FICHA DE HERRAMIENTA

=====

Nombre: _____

Tipo: ☐ Solo lectura ☐ Escritura/Acción ☐ Irreversible

Descripción para el agente:

[Esta descripción es la que verá el LLM. Debe indicar: qué hace la herramienta, cuándo usarla, cuándo NO usarla, qué devuelve.]

Parámetros:

- nombre_param (tipo): descripción

- nombre_param (tipo): descripción

Output esperado:

[Describir la estructura del resultado en caso de éxito y en caso de error]

¿Requiere punto de control antes de ejecutar? ☐ Sí ☐ No

Justificación: _____

Ejercicio 3 — Trazar el ciclo

Tarea: Usando las herramientas diseñadas en el ejercicio 2, trazar el ciclo completo del agente para el siguiente escenario:

Escenario: El cliente CLT-0231 reporta que su envío ENV-20260718-4412, con fecha de entrega prometida para el 21 de julio, nunca llegó. Hoy es 25 de julio.

Usar el siguiente formato para cada iteración:

```

--- Iteración N ---

Thought: [El razonamiento del LLM en esta iteración]

Action: herramienta(parametros)

Observation: [El resultado que devuelve la herramienta]

[Si hay punto de control]

--- Punto de control N ---

Presentación al operador: [Descripción de la acción propuesta y su justificación]

Decisión del operador: Aprueba / Rechaza / Modifica

```

El ciclo debe incluir al menos 3 iteraciones y no más de 6.

Ejercicio 4 — Anticipar fallos

Tarea: Para el agente diseñado, identificar tres escenarios de fallo y definir cómo debe responder el agente en cada uno.

Usar el formato:

```

ESCENARIO DE FALLO

=====

Descripción del fallo:

[¿Qué ocurre? ¿Qué herramienta falla? ¿Qué información está ausente?]

¿En qué iteración puede ocurrir?: ____

Respuesta esperada del agente:

[¿Qué debe hacer el agente? ¿Cómo debe reportar el fallo? ¿Qué información parcial
puede proveer? ¿Qué opciones ofrece al operador?]

¿Cómo se detecta este fallo en el diseño?:

[¿Es un error explícito de la herramienta? ¿Es una observación vacía inesperada?
¿Es un timeout? ¿Cómo distingue el agente este fallo de un resultado válido?]

```

Ejemplo de referencia: solución parcial del Ejercicio 3

Para orientar el trabajo, se incluye la traza de las primeras dos iteraciones del escenario propuesto.

--- Iteración 1 ---

Thought: El cliente reporta que el envío ENV-20260718-4412 con entrega prometida para el 21/07 no llegó. Hoy es 25/07, hay 4 días de retraso. Primero verifico el estado actual del envío para confirmar el retraso y ver si hay información de la causa.

Action: obtener_estado_envio(envio_id="ENV-20260718-4412")

Observation: {"envio_id": "ENV-20260718-4412", "estado": "en_transito", "ultima_ubicacion": "Centro de distribución Córdoba", "fecha_estimada_entrega_actualizada": "2026-07-28", "motivo_demora": "condiciones_climaticas", "cliente_id": "CLT-0231", "origen": "Buenos Aires", "destino": "Mendoza"}

--- Iteración 2 ---

Thought: El envío está efectivamente retrasado. La fecha original era 21/07, la nueva estimación es 28/07 (7 días de retraso). La causa registrada es "condiciones climáticas". Necesito verificar si la política de reclamos establece qué corresponde en este caso.

Action: buscar_politica_reclamos(causa="condiciones_climaticas", dias_retraso=7)

Observation: {"politica": "fuerza_mayor", "reembolso_aplicable": "no", "compensacion": "cupon_descuento_10_porcentaje_proximo_envio", "comunicacion_recomendada": "disculpa con explicación de causa y fecha estimada revisada"}

Criterios de evaluación

Un diseño correcto del agente debe satisfacer estos criterios:

- Las herramientas son atómicas: cada una hace una sola cosa claramente definida.
- Ninguna herramienta irreversible se ejecuta sin punto de control.
- El ciclo trazado es coherente: cada iteración usa información de las iteraciones anteriores.
- Los fallos anticipados son realistas y las respuestas son concretas y útiles para el operador.

- El límite de iteraciones está justificado para la complejidad típica del caso.

Nota del Arquitecto

El error más frecuente en este laboratorio es diseñar herramientas demasiado amplias que hacen varias cosas a la vez. La tentación es reducir el número de herramientas para simplificar el diseño. El resultado es el anti-patrón de herramientas no atómicas: más difícil de depurar, menos reutilizable y más frágil ante cambios en los sistemas externos. Si una herramienta devuelve más de un tipo de información, es una señal de que probablemente debería dividirse en dos.

Ideas clave

- El diseño de un agente requiere definir explícitamente: objetivo, herramientas, condiciones de terminación y respuestas de fallo. La implementación es secundaria a estas decisiones.
- Trazar el ciclo manualmente para un caso concreto es el método más efectivo para identificar huecos en el diseño antes de implementar.
- Los puntos de control para acciones irreversibles deben estar presentes en el diseño desde el inicio, no añadirse como corrección posterior.
- Anticipar fallos no es pesimismo: es la práctica que distingue un prototipo de un sistema de producción.

Transición hacia la siguiente sección

El laboratorio completa la parte práctica del capítulo. La siguiente sección proporciona una lista de verificación estructurada que el AI Engineer puede usar antes de desplegar cualquier agente en un entorno de producción.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 12 — Checklist del AI Engineer

"Una lista de verificación no reemplaza el juicio. Lo complementa: captura el conocimiento acumulado de lo que puede salir mal antes de que salga mal."

Propósito

Esta sección presenta la lista de verificación que el AI Engineer debe aplicar antes de desplegar un agente en un entorno de producción. Está organizada por las categorías críticas de diseño identificadas a lo largo del capítulo.

La lista no es exhaustiva para todos los contextos posibles. Es el conjunto mínimo de verificaciones que deben superarse antes de que un agente opere con usuarios o sistemas reales.

Categoría 1: Definición del agente

- [] El objetivo del agente está definido con precisión: qué debe producir, para quién y en qué condiciones.
- [] Las condiciones de éxito son verificables, no ambiguas.
- [] Las condiciones de fallo están definidas y son detectables por la capa de orquestación.
- [] El nivel de autonomía está documentado y aprobado por el responsable del sistema.
- [] Los límites del agente están establecidos en el system prompt: qué puede hacer y qué nunca debe intentar.

Categoría 2: Herramientas

- [] Cada herramienta tiene una descripción que indica cuándo usarla y cuándo no.
 - [] El formato de parámetros y el formato del output están documentados para cada herramienta.
 - [] Las herramientas irreversibles están marcadas como tales en su descripción.
 - [] No hay herramientas con efectos secundarios ocultos no documentados.
 - [] El catálogo de herramientas cubre las operaciones necesarias sin incluir herramientas innecesarias.
 - [] Existe una herramienta o mecanismo explícito para que el agente declare terminación.
-

Categoría 3: Puntos de control y supervisión

- [] Todas las acciones irreversibles tienen un punto de control que requiere confirmación humana antes de ejecutarse.
 - [] Todas las acciones que afectan a terceros (enviar comunicaciones, modificar datos de clientes) tienen punto de control.
 - [] La política de escalada está definida: cuándo el agente escala y a quién.
 - [] El agente tiene instrucciones explícitas sobre cómo comportarse en situaciones ambiguas (escalar, no adivinar).
-

Categoría 4: Condiciones de terminación y fallos

- [] Existe un límite de iteraciones implementado en la capa de orquestación (no solo en el razonamiento del LLM).
 - [] Existe un timeout absoluto de ejecución.
 - [] La capa de orquestación detecta bucles (misma acción, mismos parámetros, iteraciones consecutivas).
 - [] El agente produce una respuesta de fallo informativa cuando no puede completar el objetivo.
 - [] La respuesta de fallo incluye: qué se completó, en qué punto ocurrió el obstáculo, y las opciones disponibles.
-

Categoría 5: Gestión de contexto y memoria

- [] El contexto no crece sin límite: existe una estrategia de compresión o gestión del historial.
 - [] El límite de tokens por iteración está calculado y no supera el 70% de la ventana disponible en la iteración más costosa esperada.
 - [] La información que persiste en memoria está definida: qué se guarda y qué se descarta.
 - [] La memoria con datos sensibles tiene controles de acceso y política de retención.
-

Categoría 6: Seguridad

- [] El agente no puede actuar fuera de su alcance definido, incluso si el usuario lo solicita explícitamente.
 - [] Las herramientas que acceden a sistemas externos usan autenticación y no exponen credenciales en el contexto.
 - [] El sistema valida los parámetros generados por el LLM antes de pasarlos a las herramientas (no confía ciegamente en la generación del modelo).
 - [] Existe logging de todas las acciones del agente para auditoría posterior.
 - [] Se han probado escenarios de inyección de prompt: ¿puede un input malicioso del usuario hacer que el agente ejecute acciones no autorizadas?
-

Categoría 7: Evaluación y monitoreo

- [] Existe un conjunto de casos de prueba que cubren el happy path, casos borde y escenarios de fallo.
 - [] El número promedio de iteraciones por tarea ha sido medido en el entorno de staging.
 - [] El costo promedio por ejecución ha sido calculado y es aceptable para el volumen esperado.
 - [] Existen métricas en producción: latencia por iteración, tasa de éxito, tasa de escalada, número de iteraciones promedio.
 - [] Existe una alerta para cuando la tasa de fallos supera un umbral definido.
-

Categoría 8: Documentación operativa

- [] El system prompt del agente está bajo control de versiones.
 - [] Los cambios al system prompt tienen un proceso de revisión y testing antes de desplegarse.
 - [] Existe documentación de las herramientas disponible para el equipo de operaciones.
 - [] Existe un runbook para los fallos más frecuentes: cómo diagnosticar, cómo intervenir manualmente si el agente falla.
-

Uso de la lista

Esta lista debe completarse antes del primer despliegue a producción y revisarse antes de cada cambio significativo en el sistema (nuevas herramientas, cambios en el system prompt, cambios en la capa de orquestación).

Los ítems marcados como no aplica deben documentarse con la justificación. Un ítem que no aplica sin justificación es un ítem pendiente.

Nota del Arquitecto

Esta lista cubre el diseño del agente. La operación del agente después del despliegue requiere una lista adicional: qué monitorear, cómo interpretar las métricas, cuándo intervenir manualmente y cómo actualizar el agente sin interrumpir el servicio. La lista de diseño y la lista de operación son complementarias; una sin la otra produce sistemas que funcionan en el despliegue inicial pero degradan en las semanas siguientes.

Ideas clave

- El checklist de despliegue organiza las verificaciones en ocho categorías: definición, herramientas, puntos de control, condiciones de terminación, gestión de contexto, seguridad, evaluación y documentación.

- Los ítems de seguridad y puntos de control no son negociables antes del despliegue en producción.
 - Un ítem marcado como "no aplica" debe justificarse. La ausencia de justificación es una señal de que el ítem no fue considerado.
 - La lista de despliegue es distinta de la lista de operación. Ambas son necesarias para un sistema de agentes en producción.
-

Transición hacia la siguiente sección

Con el checklist completado, el capítulo llega a su cierre. La siguiente sección consolida las ideas principales del capítulo en un resumen estructurado que puede usarse como referencia rápida.

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 13 — Resumen del capítulo

"Un capítulo sobre agentes vale lo que vale su definición de partida. Todo lo demás es consecuencia."

Propósito de esta sección

Esta sección consolida las ideas fundamentales del capítulo en un formato de referencia rápida. Está organizada por los núcleos conceptuales del capítulo, en el mismo orden en que fueron desarrollados.

Qué es un agente de IA

Un agente de IA es un sistema que percibe un objetivo, planifica una secuencia de acciones para alcanzarlo, ejecuta esas acciones mediante herramientas, observa los resultados intermedios y adapta su plan en función de esas observaciones. La capacidad de adaptación iterativa es su característica definitoria.

La distinción entre un asistente y un agente no es de complejidad: es de estructura de control. El asistente opera en ciclo estímulo-respuesta. El agente opera en un bucle de planificación, acción y observación que puede extenderse durante múltiples iteraciones.

La autonomía de un agente es una variable de diseño, no un atributo fijo. El nivel correcto depende del riesgo de las acciones, la reversibilidad de los errores y el costo de la supervisión.

Los componentes de un agente

Seis componentes componen cualquier arquitectura de agente:

1. **Núcleo de razonamiento (LLM):** Interpreta el objetivo, genera el plan, decide las acciones y formula la respuesta final. Es el motor de todo el sistema.
 2. **Módulo de planificación:** Estructura cómo el agente aborda el objetivo, sea de forma implícita (dentro del razonamiento del LLM) o explícita (plan generado antes de la ejecución).
 3. **Módulo de estado:** Registro efímero de la ejecución actual: objetivo, acciones, observaciones, paso en curso.
 4. **Módulo de herramientas:** Los efectores del agente. Permiten actuar sobre sistemas externos. La decisión de qué herramienta usar en cada paso es parte del razonamiento del LLM.
 5. **Módulo de memoria:** Continuidad entre ejecuciones. Contiene perfil del usuario, resultados de tareas anteriores, preferencias operativas.
 6. **Capa de orquestación:** Coordina el ciclo completo: invoca al LLM, ejecuta herramientas, actualiza el estado, detecta las condiciones de terminación.
-

Arquitecturas y ciclos

Los patrones de arquitectura más estables en producción son:

- **ReAct (Reason + Act):** Ciclo de tres elementos en cada iteración: Thought (razonamiento explícito), Action (herramienta a invocar) y Observation (resultado del sistema). Es el patrón más usado y el más transparente.
- **Plan-and-Execute:** Separa planificación y ejecución. Ventajoso para tareas con pasos predefinibles.
- **Reflection:** Añade autoevaluación después de cada acción. Útil cuando el resultado es verificable automáticamente.
- **Self-Ask:** Descompone preguntas complejas en sub-preguntas. Efectivo para consultas con múltiples pasos de razonamiento encadenado.

El contexto del agente crece con cada iteración. Para tareas largas, se requieren estrategias activas de compresión del historial o uso de estado estructurado separado.

Estado, memoria y herramientas

El estado es efímero: específico a la ejecución actual. La memoria es persistente: proporciona continuidad entre sesiones. Confundir ambos produce diseños donde se guarda lo que debería descartarse o se descarta lo que debería recordarse.

Las herramientas y RAG tienen roles distintos: RAG recupera conocimiento semántico de un corpus no estructurado; las herramientas de búsqueda estructurada recuperan datos exactos de sistemas operacionales. En agentes empresariales maduros, ambos coexisten.

La calidad de las descripciones de las herramientas determina directamente la calidad de las decisiones del agente. Una descripción ambigua produce selecciones incorrectas; una descripción precisa guía al modelo hacia la herramienta correcta.

Orquestación y control

Los puntos de control para acciones irreversibles son el mecanismo que hace confiables a los agentes en contextos de negocio. Deben aplicarse a: acciones irreversibles, acciones que afectan a terceros y situaciones con información insuficiente.

Las condiciones de terminación deben estar en la capa de orquestación, no delegadas exclusivamente al razonamiento del LLM. El límite de iteraciones, el timeout y la detección de bucles son salvaguardas obligatorias.

Un agente que falla debe terminar gracefully, reportando el estado del proceso y las opciones disponibles para el usuario o el operador.

Patrones y anti-patrones

Los patrones que producen agentes robustos son: herramientas atómicas, verificación antes de acciones irreversibles, terminación explícita y fallback con información parcial.

Los anti-patrones más frecuentes son: el agente todo-en-uno (catálogo de herramientas sobredimensionado), la confianza ciega en el output de herramientas, la improvisación de herramientas fuera de su diseño, el contexto sin gestión y los efectos secundarios ocultos en herramientas declaradas como solo lectura.

La mayoría de los anti-patrones no aparecen en el prototipo inicial: aparecen cuando el sistema escala. El diseño debe anticiparlos.

El agente en el contexto del módulo

El agente es la arquitectura que integra los bloques construidos en los capítulos anteriores:

- Las instrucciones del sistema (capítulo 05) definen la política y los límites del agente.
- La memoria (capítulo 04) proporciona continuidad entre sesiones.
- RAG (capítulo 06) incorpora conocimiento externo al ciclo de acción.
- Las herramientas (capítulo 07) son los efectores que permiten al agente actuar.

El agente individual de este capítulo escala al sistema multiagente del capítulo 09 cuando la tarea supera la capacidad de un único agente o requiere paralelismo y especialización.

Ideas clave del capítulo

- Un agente es la síntesis de los bloques del módulo: memoria, instrucciones, RAG y herramientas integrados en un ciclo de control adaptativo.
 - ReAct es el patrón de referencia para la mayoría de los agentes empresariales. Su transparencia (razonamiento explícito) es su principal ventaja operativa.
 - La autonomía controlada —con puntos de control en acciones sensibles y condiciones de terminación robustas— es lo que distingue los agentes de producción de los agentes de demo.
 - El diseño del catálogo de herramientas es tan importante como el diseño del propio agente. Las herramientas mal descritas producen agentes que fallan silenciosamente.
-

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 14 — Autoevaluación

"La autoevaluación no mide cuánto se leyó. Mide cuánto se comprendió lo suficiente como para aplicarlo."

Propósito de la autoevaluación

Esta sección presenta preguntas para verificar la comprensión de los conceptos principales del capítulo. Las preguntas están organizadas en tres niveles: comprensión conceptual, análisis de casos y decisión de diseño. Las respuestas aparecen al final de la sección.

Nivel 1 — Comprensión conceptual

Pregunta 1. ¿Cuál es la característica que distingue fundamentalmente a un agente de IA de un asistente conversacional que usa herramientas?

- a) El agente usa modelos de lenguaje más grandes.
- b) El agente puede adaptar su plan en función de los resultados intermedios de sus acciones.
- c) El agente tiene acceso a más herramientas simultáneamente.
- d) El agente no requiere intervención humana en ningún punto del proceso.

Pregunta 2. En el ciclo ReAct, ¿cuál de los tres elementos (Thought, Action, Observation) es generado por el sistema en lugar de por el LLM?

- a) Thought.
 - b) Action.
 - c) Observation.
 - d) Los tres son generados por el LLM.
-

Pregunta 3. ¿Cuál es la diferencia principal entre el estado del agente y su memoria persistente?

- a) El estado contiene información del usuario; la memoria contiene el historial de acciones.
 - b) El estado es efímero y específico a la ejecución actual; la memoria proporciona continuidad entre ejecuciones.
 - c) El estado se almacena en la base de datos; la memoria vive en el contexto del LLM.
 - d) El estado es gestionado por el LLM; la memoria es gestionada por la capa de orquestación.
-

Pregunta 4. ¿Por qué las condiciones de terminación de un agente deben estar implementadas en la capa de orquestación y no solo en el razonamiento del LLM?

- a) Porque el LLM no puede generar texto de terminación.
 - b) Porque el LLM puede generar bucles o condiciones de fallo que no detecta por sí mismo.
 - c) Porque la capa de orquestación tiene acceso a las herramientas y el LLM no.
 - d) Porque las condiciones de terminación son costosas computacionalmente.
-

Nivel 2 — Análisis de casos

Pregunta 5. Un equipo diseña un agente de análisis financiero con una herramienta llamada `generar_reporte(cliente_id, periodo)`. La herramienta genera el reporte, lo envía por email al cliente y registra la generación en el sistema de auditoría. ¿Qué anti-patrón describe mejor este diseño?

- a) El agente todo-en-uno.
- b) Herramienta con efectos secundarios ocultos (no atómica).
- c) Confianza ciega en el output de herramientas.
- d) Improvisación de herramientas.

Pregunta 6. Un agente ejecuta la misma acción `buscar_pedidos(cliente="CLT-0231", estado="pendiente")` en las iteraciones 4, 5 y 6 sin cambiar los parámetros. ¿Qué mecanismo de la capa de orquestación debería activarse?

- a) Punto de control para confirmación humana.
- b) Detección de bucle y terminación con reporte de estado.
- c) Compresión del historial de contexto.
- d) Escalada automática al siguiente nivel de soporte.

Pregunta 7. Un agente de soporte tiene que verificar si un cliente tiene contratos vencidos. Tiene disponibles dos herramientas: `buscar_contratos(cliente_id, estado)` y `buscar_en_documentacion(consulta)`. ¿Cuál debe usar y por qué?

- a) `buscar_en_documentacion` porque la consulta es semántica.
- b) `buscar_contratos` porque la consulta es estructurada y el ID del cliente está disponible.
- c) Ambas en paralelo para aumentar el recall.
- d) Ninguna: el agente debería razonar sobre los contratos usando su conocimiento interno.

Nivel 3 — Decisión de diseño

Pregunta 8. Un agente empresarial debe enviar un email de oferta personalizada a un cliente como paso final de su ciclo. ¿Qué considera el AI Engineer al diseñar esta acción?

Responda desarrollando brevemente: (a) si esta acción requiere punto de control, (b) qué información debe presentarse al operador antes de la confirmación, y (c) qué debe hacer el agente si el operador rechaza la acción.

Pregunta 9. Un equipo observa que su agente funciona correctamente en staging (100 tickets de prueba) pero en producción, después de una semana, presenta un 15% de tickets con más de 10 iteraciones. El límite configurado es 15 iteraciones. ¿Qué tres causas probables investigaría el AI Engineer y cómo?

Pregunta 10. Un agente debe procesar solicitudes de crédito. El objetivo es: verificar el historial crediticio del solicitante, consultar la política de aprobación vigente, y emitir una recomendación (aprobar, rechazar, revisar manualmente). Diseñar brevemente: el nivel de autonomía apropiado, los puntos de control necesarios y el sistema de fallback si la herramienta de historial crediticio no responde.

Respuestas

Respuesta 1: (b). La capacidad de adaptar el plan en función de los resultados intermedios es la característica definitoria del agente. El tamaño del modelo, el número de herramientas y el nivel de autonomía son variables de diseño, no atributos definitorios.

Respuesta 2: (c). La Observation es el resultado generado por el sistema al ejecutar la herramienta indicada en la Action. El LLM genera el Thought y la Action; el sistema genera la Observation.

Respuesta 3: (b). El estado es efímero y registra lo que está ocurriendo en la ejecución actual. La memoria persiste entre ejecuciones y proporciona continuidad. Son conceptos distintos con roles distintos en la arquitectura.

Respuesta 4: (b). El LLM puede entrar en bucles de razonamiento donde no detecta que está bloqueado, o puede malinterpretar el estado del proceso y no generar la señal de terminación correcta. La capa de orquestación implementa salvaguardas externas al razonamiento del modelo.

Respuesta 5: (b). La herramienta hace tres cosas: genera el reporte, lo envía por email y registra en auditoría. No es atómica. Además, el envío del email es un efecto secundario con consecuencias en el mundo externo que el agente puede no anticipar si invoca la herramienta para "solo generar" el reporte.

Respuesta 6: (b). La repetición de la misma acción con los mismos parámetros en iteraciones consecutivas es un indicador de bucle. La capa de orquestación debe detectar este patrón y terminar el ciclo con un reporte del estado en que quedó el proceso.

Respuesta 7: (b). La consulta tiene un parámetro exacto (cliente_id) y busca datos estructurados (contratos con un estado específico). `buscar_contratos` es la herramienta apropiada. `buscar_en_documentacion` (RAG) es la elección correcta para recuperar políticas o

procedimientos por contenido semántico, no para recuperar datos operacionales de un cliente específico.

Respuesta 8: (a) Sí requiere punto de control, porque el envío de un email a un cliente es irreversible y afecta a un tercero. (b) El operador debe ver: el destinatario del email, el asunto, el cuerpo completo del mensaje, la información del cliente en que se basa la personalización, y el contexto del ciclo que llevó a esta acción. (c) Si el operador rechaza, el agente no debe reintentar automáticamente. Debe registrar el rechazo, reportar al operador qué quedó pendiente (la oferta no fue enviada) y ofrecer opciones: reformular el mensaje, cancelar la tarea o escalar.

Respuesta 9: Tres causas probables: (1) Aparición de tickets de tipos no cubiertos en los casos de prueba, lo que hace que el agente explore más iteraciones buscando herramientas o información no disponible. Diagnóstico: analizar el razonamiento de los tickets con más de 10 iteraciones para identificar patrones comunes. (2) Una herramienta que en staging respondía rápido en producción devuelve resultados incompletos o parciales, haciendo que el agente reintente. Diagnóstico: verificar las tasas de error y tiempo de respuesta de cada herramienta en producción. (3) Tickets con ambigüedad en el objetivo: el agente no puede determinar cuándo ha completado el objetivo y sigue iterando. Diagnóstico: revisar el system prompt y agregar criterios de terminación más explícitos para los casos problemáticos.

Respuesta 10: Nivel de autonomía semi-autónomo: el agente puede consultar herramientas y analizar autónomamente, pero la recomendación final (aprobar o rechazar) requiere punto de control antes de ser registrada. Puntos de control necesarios: (1) antes de emitir una recomendación de aprobación o rechazo (acción con consecuencias financieras y legales), (2) si el historial crediticio indica condiciones que la política no cubre explícitamente. Fallback si la herramienta de historial no responde: el agente no debe emitir una recomendación basada en información incompleta. Debe reportar: "No se pudo completar el análisis porque el sistema de historial crediticio no respondió después de [N] intentos. La solicitud de crédito de [cliente] queda pendiente de análisis manual. Información disponible: [datos que sí se obtuvieron]."

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."

Módulo 3 — Context Engineering

Capítulo 08 — Agentes de IA: Arquitectura y Orquestación

Sección 15 — Transición al Capítulo 9

"Un agente resuelve lo que un único modelo no puede resolver solo. Un sistema multiagente resuelve lo que un único agente no puede resolver solo. La escala del problema define la escala de la solución."

Lo que el capítulo construyó

Este capítulo cubrió la arquitectura del agente individual desde su definición hasta su despliegue. El recorrido fue:

- Definir qué hace que un sistema sea agéntico: la adaptación iterativa del plan según los resultados observados.
- Identificar los seis componentes que componen cualquier agente: razonamiento, planificación, estado, herramientas, memoria y orquestación.
- Estudiar los patrones de arquitectura estables en producción: ReAct como patrón de referencia, y sus variantes para casos específicos.
- Comprender el ciclo de percepción, planificación y acción en detalle, con la trazabilidad completa de un ciclo de tres iteraciones.
- Gestionar el estado efímero de la ejecución y la memoria persistente entre sesiones.
- Coordinar herramientas y RAG como los dos mecanismos principales de acción y recuperación.
- Diseñar la orquestación interna: cuándo actuar autónomamente, cuándo escalar, cuándo declarar terminación.
- Reconocer los patrones que producen sistemas robustos y los anti-patrones que explican los fallos más frecuentes.

- Aplicar todo lo anterior a un caso de estudio empresarial completo y un laboratorio de diseño.

Con este capítulo, el lector puede diseñar, trazar y evaluar agentes de IA individuales en contextos empresariales reales.

El límite del agente individual

El agente individual tiene límites inherentes a su arquitectura:

Límite de complejidad. Una tarea que requiere mantener simultáneamente docenas de hilos de razonamiento independientes puede superar la capacidad cognitiva de un único ciclo ReAct. El estado crece, el contexto se vuelve difícil de gestionar, y el razonamiento se degrada.

Límite de paralelismo. Un agente individual ejecuta sus acciones de forma secuencial (o con paralelismo limitado en algunas implementaciones). Tareas que se beneficiarían de ejecución paralela de subtareas independientes no pueden aprovecharlo en un diseño de agente único.

Límite de especialización. Un agente diseñado para ser generalista inevitablemente es subóptimo en dominios específicos. Un agente diseñado para un dominio específico no puede manejar bien los demás. La especialización y la generalidad son objetivos en tensión dentro de un único agente.

Límite de confianza. En algunas aplicaciones, es deseable que diferentes partes del proceso operen con diferentes niveles de privilegio o con capacidades de revisión mutua. Un único agente no puede revisarse a sí mismo de forma independiente.

Cuándo el agente individual no es suficiente

Existen categorías de problemas donde un sistema multiagente es la respuesta correcta:

Tareas que requieren perspectivas independientes. Generar un análisis y luego criticar ese análisis de forma independiente. Un agente que se autoevalúa puede sesgar la crítica hacia validar su propio trabajo. Dos agentes independientes producen una revisión más confiable.

Tareas con subtareas verdaderamente paralelas. Analizar simultáneamente tres mercados distintos para sintetizar una recomendación de expansión. Las tres tareas de análisis son independientes entre sí y pueden ejecutarse en paralelo. Un sistema multiagente puede reducir significativamente la latencia total.

Tareas con dominios claramente distintos. Un proceso que requiere expertise legal, expertise técnico y expertise comercial puede beneficiarse de agentes especializados en cada dominio, coordinados por un agente orquestador que sintetiza los resultados.

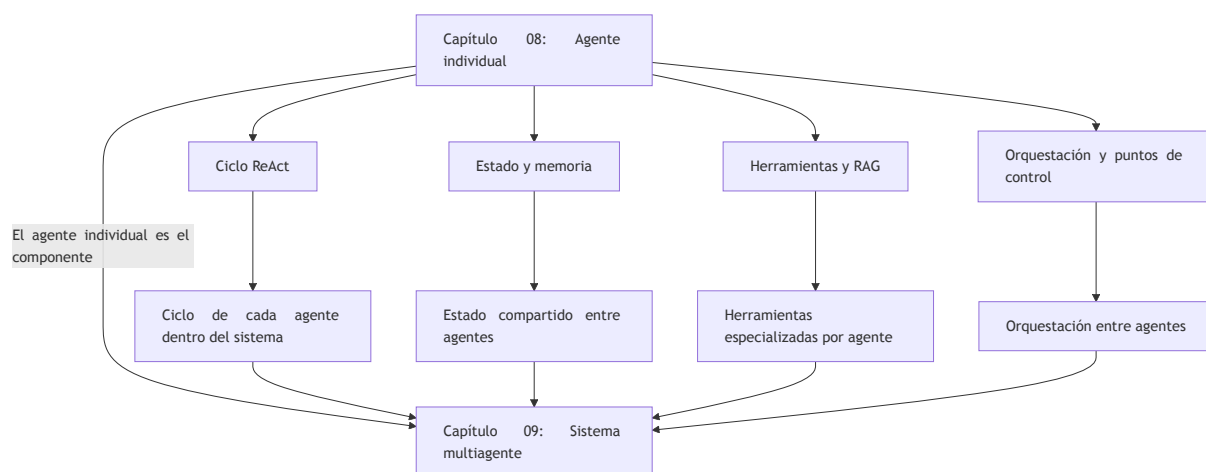
Tareas de larga duración con múltiples fases. Un proyecto de análisis que toma días o semanas puede estructurarse como una red de agentes donde cada uno maneja una fase, con transferencias de estado explícitas entre fases.

El salto al capítulo 9

El capítulo 09 estudia sistemas multiagente: arquitecturas donde múltiples agentes individuales se coordinan para resolver tareas que superan la capacidad de uno solo.

Los conceptos de este capítulo son la base de ese estudio:

- El agente individual del capítulo 08 es el componente del sistema multiagente del capítulo 09.
- Los mismos patrones (ReAct, estado, herramientas, puntos de control) se aplican a cada agente individual dentro del sistema.
- Los nuevos desafíos del capítulo 09 son los desafíos de coordinación: cómo los agentes se comunican, cómo se delegan trabajo, cómo mantienen coherencia colectiva y cómo se gestiona el fallo de un agente individual sin que colapse el sistema completo.



Una pregunta de diseño para llevar

Antes de leer el capítulo 09, reflexionar sobre este escenario:

Una empresa quiere construir un sistema de IA que analice contratos legales complejos. El análisis requiere identificar cláusulas de riesgo, verificar el cumplimiento regulatorio, comparar con contratos anteriores del mismo cliente y producir un informe ejecutivo para el área de negocio.

¿Es este un problema que puede resolver un agente individual bien diseñado, o requiere un sistema multiagente? ¿Qué factores determinan esa decisión?

No hay una respuesta única. Los factores que importan son: la longitud típica de los contratos, el volumen de contratos por período, los plazos de respuesta esperados, la disponibilidad de expertise especializado que justifique agentes distintos, y la complejidad de coordinación que un sistema multiagente introduce. El capítulo 09 dará las herramientas para responder esta pregunta con criterios técnicos precisos.

Ideas clave de la transición

- El agente individual del capítulo 08 tiene límites de complejidad, paralelismo, especialización y revisión independiente. Esos límites definen cuándo es necesario escalar a sistemas multiagente.
 - El sistema multiagente del capítulo 09 está compuesto por agentes individuales. Los conceptos de este capítulo son sus bloques de construcción.
 - Los nuevos desafíos del capítulo 09 son desafíos de coordinación: comunicación, delegación, coherencia colectiva y resiliencia ante fallos individuales.
-

"Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones."